CIE-1 QUESTION BANK · 5TH SEMESTER · CS51

Most Likely 5-Mark Questions

5 MARKS

SOFTWARE ENGINEERING & MODELLING

23 questions across all three CIE-1 units, each with a **model answer you can write in 10 minutes**, chosen using last year's CIE-1 paper and the syllabus.

- Model answers with diagrams
- Marks split for each answer
- Last year's paper mapped
- 2 predicted papers

UNIT 1 · 8 QUESTIONS

Process & Agile

- Software engineering, attributes of good software, key challenges
- ACM/IEEE Code of Ethics and issues of professional responsibility
- Waterfall model: diagram, when appropriate, problems
- Incremental development: diagram, advantages, problems
- Coping with change: prototyping and incremental delivery

UNIT 2 · 8 QUESTIONS

Requirements & Design

- Functional vs non-functional requirements
- NFRs: types and metrics
- Spiral RE process
- Elicitation & analysis; interviews, ethnography
- Structure of the requirements document

UNIT 3 · 7 QUESTIONS

Architecture (CIE-1 part)

- What is architectural design? Useful views
- Krutchen's 4+1 view model with a diagram
- Architectural design decisions: the 8 fundamental questions
- How non-functional requirements shape the architecture
- Advantages of explicit architecture; architecture in the small vs large

Companion to the **CIE-1 Study Guide**. Built from your lecture slides, **Sommerville (10th ed)** and **Pressman**.

Course coordinators
Mr. Pradeep Kumar D · Dr Shilpa S Chaudhari

Read this first

This booklet gives you the **5-mark questions most likely to appear in CIE-1**, 7–8 per unit, each with a **model answer written the way you should write it in about 10 minutes**: definition, a diagram you can draw quickly, and numbered points with keywords. Each unit ends with backup questions, and the last section is a predicted CIE-1 paper.

The exam pattern (from last year's CIE–1 paper)

Part	What you do	Marks
Q1 (compulsory)	Answer **all three** parts: 1a, 1b, 1c. One part comes from **each unit** (CO1, CO2, CO3).	3 × 5 = 15
Q2 *or* Q3	Choose **one** question and answer all three of its parts. Parts mix Units 1 and 2.	3 × 5 = 15
Total	1 hour, so about **10 minutes per 5-mark answer**, roughly one page each.	30

What last year's paper tells us

Unit 3 is guaranteed

Q1 is compulsory and always has a CO3 part. Unit 3's CIE-1 portion is small (7 questions here), so it is the **easiest guaranteed 5 marks**.

Units 1 & 2 carry the rest

In 2024: **4 parts from Unit 1**, **4 from Unit 2**, **1 from Unit 3**. Prepare Units 1 and 2 well enough to have a free choice between Q2 and Q3.

Diagrams score

Every part was "explain/describe" (Bloom L1–L2), and **3 of 9 demanded a neat labelled diagram** (RE spiral, Scrum sprint cycle, incremental model).

How the questions were chosen

No one can guarantee a question. These are ranked using last year's paper (teachers often repeat), the exact syllabus lines, and the classic diagram questions in this subject. The tags tell you how sure we are:

ASKED IN CIE-1 2024 · VERY LIKELY · LIKELY

✎ ORDER OF PREPARATION

1) every **Asked in 2024** question → 2) every **Very likely** → 3) **Likely** → 4) backup questions. Draw each diagram once on paper; reading it is not enough.

Last year's paper, mapped to this booklet

Q	2024 question (5 marks each)	CO	Answer in this booklet
1a	What is software engineering? Attributes of good software? Challenges faced in SE?	CO1	Unit 1 · Q1
1b	Requirements engineering process with a neat labelled spiral diagram.	CO2	Unit 2 · Q3
1c	What is software architecture design? What views or perspectives are useful?	CO3	Unit 3 · Q1
2a	Advantages of XP practices: refactoring, pair programming, user stories.	CO1	Unit 1 · Q7

Q	2024 question (5 marks each)	CO	Answer in this booklet
2b	Principles of agile; Scrum sprint cycle with a neat labelled diagram.	CO1	Unit 1 · Q8
2c	Non-functional requirements: types and metrics for specifying them.	CO2	Unit 2 · Q2
3a	Structure of the IEEE software requirements specification document.	CO2	Unit 2 · Q5
3b	System modelling; sequence diagrams in interaction models.	CO2	Not in your current syllabus (see note)
3c	Incremental development model with a neat diagram; advantages and problems.	CO1	Unit 1 · Q4

⚠ **ABOUT LAST YEAR'S 3B (SYSTEM MODELLING)**

Last year's Unit 2 included **system modelling** (Sommerville Ch 5). Your syllabus copy replaces it with **Design Concepts** (Pressman), so it isn't in this booklet. If your teacher says system modelling is still part of CIE-1, ask for it and it can be added.

The 10-minute answer method

1. **1 minute: plan.** Write the definition and list your point headings in the margin.
2. **3 minutes: draw.** Draw the diagram first, neat and labelled, with a caption.
3. **5 minutes: write.** Numbered points with a **bold or underlined keyword** and one explaining line each.
4. **1 minute: check.** Did you answer every part of the question (for example "advantages *and* problems")?

Contents

UNIT 1 Process & Agile 4

Q1 What is software engineering? What are the essential attributes of good software? List the key ch... 4
Q2 Explain the ACM/IEEE Code of Ethics. What are the issues of professional responsibility? 5
Q3 Explain the waterfall model with a neat diagram. When is it appropriate, and what are its problems? 7
Q4 With a neat diagram describe the incremental development model. What are the advantages and ... 8
Q5 How do software processes cope with change? Explain software prototyping (with the process of... 9
Q6 Explain the process improvement cycle and the SEI capability maturity levels with a neat diagram. 11
Q7 Describe and explain the advantages of the following XP practices: (i) refactoring (ii) pair progra... 12
Q8 List the principles of agile methods and explain the Scrum sprint cycle with a neat labelled diagram. 14
Backup questions (prepare if you have time) 15

UNIT 2 Requirements & Design 16

Q1 Differentiate between functional and non-functional requirements with examples. 16
Q2 What are non-functional requirements? List the types of non-functional requirements. What are t... 17
Q3 Explain the requirements engineering process with a neat labelled spiral diagram. 19
Q4 Explain the requirements elicitation and analysis process with a neat diagram. Briefly explain inter... 20
Q5 Explain the structure of the IEEE software requirements specification (requirements document). 21
Q6 What is requirements validation? Explain the requirements checks and validation techniques. 23
Q7 Explain the following design concepts: abstraction, modularity, information hiding and functional i... 24
Q8 Explain how the requirements model is translated into the design model, and describe the elemen... 25
Backup questions (prepare if you have time) 26

UNIT 3 Architecture (CIE–1 part) 28

Q1 What is software architecture design? What views or perspectives are useful when designing and ... 28
Q2 Explain Krutchen's 4+1 view model of software architecture with a neat diagram. 29
Q3 What are architectural design decisions? Explain the fundamental questions an architect must an... 31
Q4 Explain how architectural design depends on the non-functional requirements of a system (perfor... 32
Q5 What are the advantages of designing and documenting the software architecture explicitly? Diffe... 33
Q6 What are box-and-line diagrams? Explain the two ways in which architectural models are used. 34
Q7 Explain architectural design with the example of the packing robot control system. Why must the ... 35
Backup questions (prepare if you have time) 36

PRACTICE Predicted CIE-1 papers A and B 37

UNIT 1 · MOST LIKELY 5-MARK QUESTIONS

Unit 1: 8 questions you must prepare

CO1 · Software engineering and ethics · Process models · Coping with change · Process improvement · Agile, XP and Scrum

QUESTIONS	FROM	TIME PER ANSWER
8 model answers + 5 backups	Study guide Chapters 1, 2 and 3	About 10 minutes in the exam

QUESTIONS AT A GLANCE

1. Software engineering, attributes of good software, key challenges (asked 2024)
2. ACM/IEEE Code of Ethics and issues of professional responsibility
3. Waterfall model: diagram, when appropriate, problems
4. Incremental development: diagram, advantages, problems (asked 2024)
5. Coping with change: prototyping and incremental delivery
6. Process improvement cycle and SEI capability maturity levels
7. XP practices: refactoring, pair programming, user stories (asked 2024)
8. Agile principles and the Scrum sprint cycle (asked 2024)

Q1 What is software engineering? What are the essential attributes of good software? List the key challenges faced in software engineering.

5 MARKS · CO1 · L2 · ASKED IN CIE-1 2024 · VERY LIKELY

Marks split → Definition 1 + Four attributes 2 + Challenges 2

MODEL ANSWER

DEFINITION — LEARN THIS LINE

Software engineering is an engineering discipline that is concerned with all aspects of software production, from the early stages of system specification through to maintaining the system after it has gone into use.

- **Engineering discipline**: using appropriate theories and methods to solve problems, bearing in mind organisational and financial constraints.
- **All aspects of software production**: not just the technical process of development, but also project management and the development of tools, methods and theories. (*Software* = computer programs and associated documentation.)

Essential attributes of good software. Good software should deliver the required functionality and performance to the user and should be maintainable, dependable and usable:

1. **Maintainability**: written so that it can evolve to meet the changing needs of customers, because software change is inevitable in a changing business environment.

2. **Dependability and security**: includes reliability, security and safety; it should not cause physical or economic damage if the system fails, and malicious users should not be able to access or damage it.
3. **Efficiency**: no wasteful use of system resources such as memory and processor cycles (responsiveness, processing time, memory utilisation).
4. **Acceptability**: acceptable to the type of users it is designed for, i.e. understandable, usable and compatible with the other systems they use.

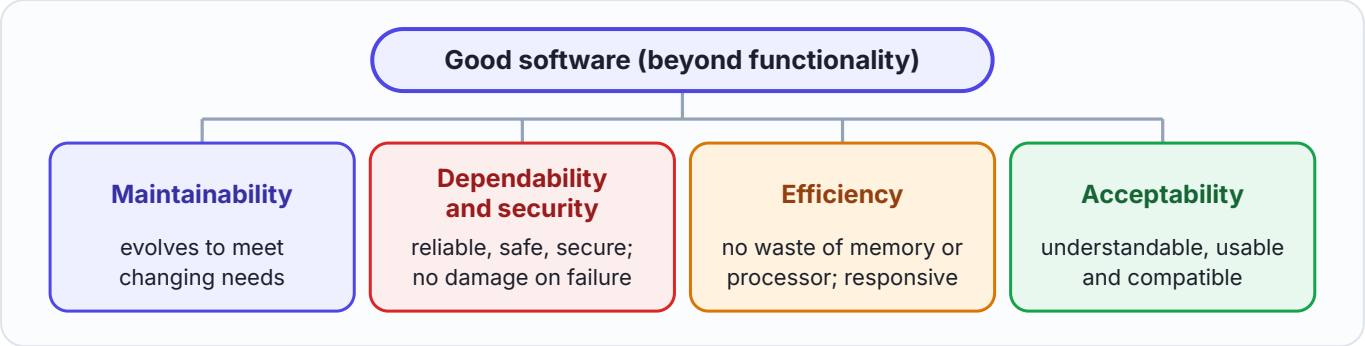


Fig Q1 The four essential attributes of good software (quick sketch for the answer). Sommerville Fig 1.2 · Study guide 1.4

Key challenges. Four general issues affect many types of software and create its main challenges:

Challenge	Meaning
1. Heterogeneity	Systems must operate as distributed systems across networks of different types of computers and mobile devices, and integrate with older legacy systems.
2. Business and social change	Business and society change very quickly, so existing software must be changed and new software developed rapidly.
3. Security and trust	Software is intertwined with all aspects of our lives, so we must be able to trust it, especially remote systems used through web pages or web services.
4. Scale	Software is built at very different scales, from very small embedded systems in wearable devices to Internet-scale, cloud-based systems serving a global community.

In short, the key challenges are **coping with increasing diversity, demands for reduced delivery times and developing trustworthy software**.

> **MEMORY TRICK**
> **"My Dog Eats Apples"** = **M**aintainability, **D**ependability & security, **E**fficiency, **A**cceptability. **"Hungry Bears Steal Snacks"** = **H**eterogeneity, **B**usiness & social change, **S**ecurity & trust, **S**cale.

Q2 Explain the ACM/IEEE Code of Ethics. What are the issues of professional responsibility?

5 MARKS · CO1 · L2 · LIKELY

Marks split → Ethics intro 1 + Professional responsibility 1.5 + Code and eight principles 2.5

MODEL ANSWER

Software engineering involves **wider responsibilities than simply the application of technical skills**. Software engineers must behave in an honest and ethically responsible way if they are to be respected as professionals. Ethical behaviour is **more than simply upholding the law**: it involves following a set of

principles that are morally correct.

Issues of professional responsibility (areas where acceptable behaviour is set by professional responsibility, not by laws):

1. **Confidentiality**: respect the confidentiality of employers or clients, whether or not a formal confidentiality agreement has been signed.
2. **Competence**: do not misrepresent your level of competence or knowingly accept work that is outside your competence.
3. **Intellectual property rights**: be aware of local laws on the use of intellectual property such as patents and copyright, and protect the intellectual property of employers and clients.
4. **Computer misuse**: do not use technical skills to misuse other people's computers. Misuse ranges from trivial (game playing on an employer's machine) to extremely serious (dissemination of viruses).

ACM/IEEE Code of Ethics. The professional societies in the US, **ACM and IEEE**, cooperated to produce the *Software Engineering Code of Ethics and Professional Practice*. Members of these organisations **sign up to the code** when they join. Because software engineers have significant opportunities to do good or cause harm, they commit to making software engineering a beneficial and respected profession. The code contains **eight principles** on the behaviour and decisions of professional software engineers, including practitioners, educators, managers, supervisors, policy makers, trainees and students. In accordance with their commitment to the **health, safety and welfare of the public**, software engineers shall adhere to:

Principle	Software engineers shall ...
1. Public	act consistently with the public interest.
2. Client and employer	act in a manner that is in the best interests of their client and employer consistent with the public interest.
3. Product	ensure that their products and related modifications meet the highest professional standards possible.
4. Judgment	maintain integrity and independence in their professional judgment.
5. Management	(managers and leaders) subscribe to and promote an ethical approach to the management of software development and maintenance.
6. Profession	advance the integrity and reputation of the profession consistent with the public interest.
7. Colleagues	be fair to and supportive of their colleagues.
8. Self	participate in lifelong learning regarding the practice of their profession and promote an ethical approach to the practice of the profession.

MEMORY TRICK

Principles in pairs: PC · PJ · MP · CS, "A **PC** in **PJ**s meets an **MP** from **CS**" = **P**ublic–**C**lient, **P**roduct–**J**udgment, **M**anagement–**P**rofession, **C**olleagues–**S**elf. **Responsibility issues: "Careful Coders Inspect Code"** = Confidentiality, Competence, IPR, Computer misuse.

Q3 Explain the waterfall model with a neat diagram. When is it appropriate, and what are its problems?

5 MARKS | CO1 | L2 | VERY LIKELY

Marks split → Diagram 2 + Five phases 1.5 + When appropriate and problems 1.5

MODEL ANSWER

The **waterfall model** is a **plan-driven** process model with **separate and distinct phases of specification and development**: all process activities are planned and scheduled before development starts, and each phase must be complete before the next begins. It was the first published process model (Royce, 1970) and is also called the *software life cycle*.

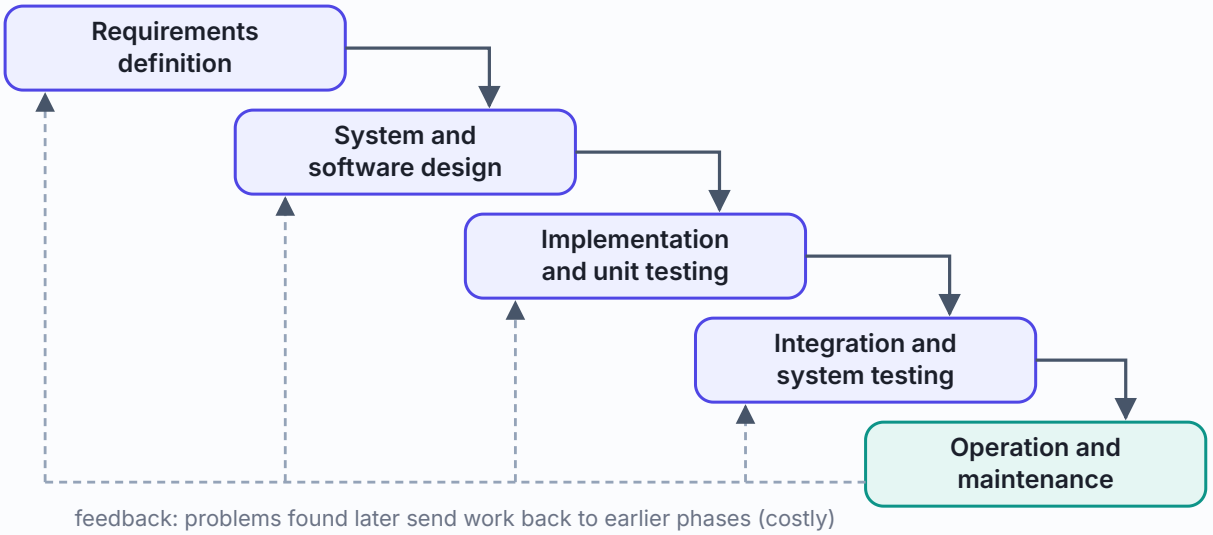


Fig Q3 The waterfall model: draw it as a staircase with dashed feedback arrows. Sommerville Fig 2.1 · Study guide Fig 2.1

1. **Requirements analysis and definition**: the system's services, constraints and goals are established with users and defined in detail as the system specification.
2. **System and software design**: requirements are allocated to hardware or software, an overall architecture is established, and the fundamental software abstractions and their relationships are identified.
3. **Implementation and unit testing**: the design is realised as a set of programs or program units; each unit is verified against its specification.
4. **Integration and system testing**: units are integrated and tested as a complete system against the requirements, then delivered to the customer.
5. **Operation and maintenance**: normally the longest phase; the system is used, and maintenance corrects errors, improves the implementation and adds new services.

The result of each phase is one or more approved (*signed-off*) documents. In practice the phases overlap and feed information back (dashed arrows). **The main drawback is the difficulty of accommodating change after the process is underway.**

WHEN IT IS APPROPRIATE

- ✓ Requirements are **well understood** and changes will be fairly limited
- ✓ **Embedded systems**: software must interface with inflexible hardware, so decisions cannot be delayed
- ✓ **Critical systems**: safety and security analysis needs a complete specification and design (e.g. insulin pump)
- ✓ **Large systems engineering projects developed at several sites**: the plan-driven nature helps coordinate the work

PROBLEMS

- ✗ **Inflexible partitioning** of the project into distinct stages makes it difficult to respond to changing customer requirements
- ✗ Few business systems have **stable requirements**, so the model suits few of them
- ✗ Requirements are **frozen early**, so the system may not do what users want
- ✗ **No working version until late**; customers find it hard to state all requirements at the start

✎ EXAM TIP

Marks go to the **labelled diagram with feedback arrows**, one line per phase, the words **"plan-driven"** and **"signed-off documents"**, and an example system for "when appropriate".

Q4 With a neat diagram describe the incremental development model. What are the advantages and problems of the incremental approach?

5 MARKS | CO1 | L2 | ASKED IN CIE-1 2024 | VERY LIKELY

Marks split → Definition 1 + Diagram 2 + Advantages and problems 2

MODEL ANSWER

DEFINITION — LEARN THIS LINE

Incremental development is based on the idea of developing an initial implementation, getting feedback from users and evolving the software through several versions until the required system has been developed. Specification, development and validation are **interleaved** rather than separate, with rapid feedback across activities.

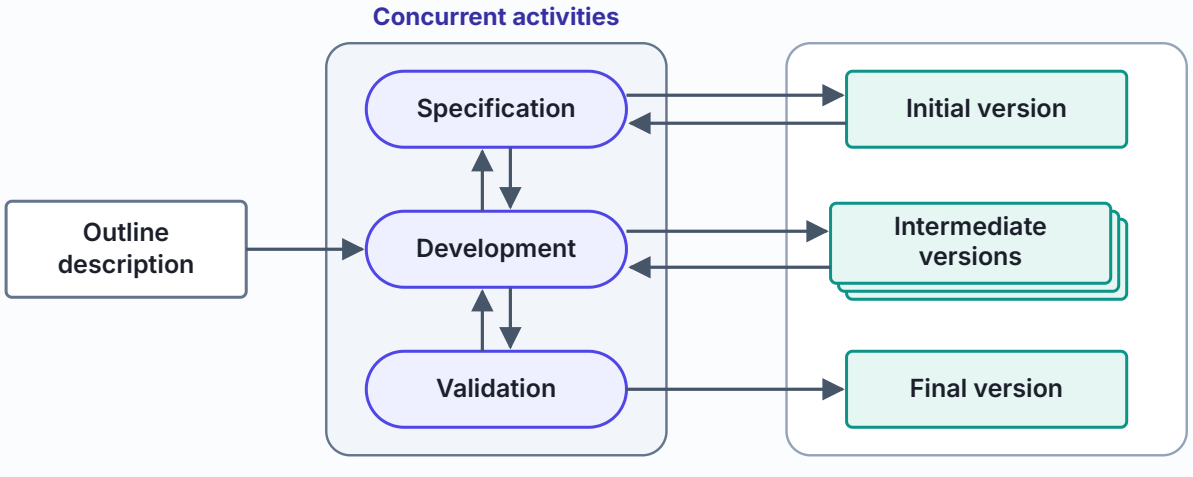


Fig Q4 Incremental development: concurrent activities produce a series of versions. Sommerville Fig 2.2 · Study guide Fig 2.2

- Starting from an **outline description**, specification, development and validation run **concurrently** and produce an **initial version**, **intermediate versions** and finally the **final version**.
- Each increment adds functionality. **Early increments include the most important or most urgently required functionality**, so the customer can evaluate the system early; if it is wrong, only the current increment has to be changed.
- It is now the **most common approach** for application systems and software products and the basis of agile methods. It may be plan-driven (increments identified in advance), agile, or a mixture.

ADVANTAGES

- ✓ **Cost of accommodating changing customer requirements is reduced**: far less analysis and documentation to redo than with waterfall
- ✓ **Easier to get customer feedback** on the work done: customers comment on demonstrations and see how much has been implemented
- ✓ **More rapid delivery and deployment of useful software**, so customers gain value earlier

PROBLEMS

- ✗ **The process is not visible**: managers need regular deliverables to measure progress, and documenting every quickly built version is not cost-effective
- ✗ **System structure tends to degrade** as new increments are added; unless time and money is spent on refactoring, regular change corrupts the structure and further changes become difficult and costly

These problems are most serious for **large, long-lifetime systems** developed by several teams, which need a stable architecture planned in advance.

> ✎ **EXAM TIP**
> Do not confuse this with **incremental delivery**: in incremental *development* an increment may only be demonstrated to the customer for comment; in incremental *delivery* each increment is deployed for real use.

Q5 How do software processes cope with change? Explain software prototyping (with the process of prototype development) and incremental delivery.

5 MARKS | CO1 | L2 | LIKELY

Marks split → Two approaches 1 + Prototyping with diagram 2 + Incremental delivery 2

MODEL ANSWER

Change is inevitable in all large software projects: business changes lead to new and changed requirements, new technologies open up new possibilities, and changing platforms require application changes. Change leads to **rework**, so processes reduce its cost in two ways:

- **Change anticipation**: the process includes activities that can anticipate possible changes before significant rework is required. Technique: **system prototyping**.
- **Change tolerance**: the process is designed so that changes can be accommodated at relatively low cost. Technique: **incremental delivery** (changes go into increments not yet developed).

(a) Software prototyping. A **prototype** is an initial version of a system used to demonstrate concepts and try out design options. It is used in **requirements engineering** (elicitation and validation), in **design** (to explore options and develop the user interface) and in **testing** (back-to-back tests).

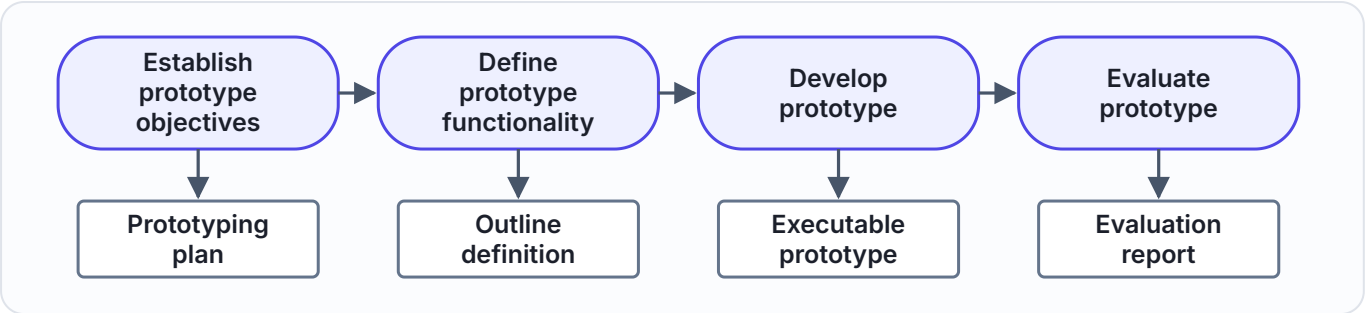


Fig Q5a The process of prototype development (activities and their outputs). Sommerville Fig 2.9 · Study guide Fig 2.12

Prototype development may use rapid prototyping tools and **leaves out functionality**: it focuses on areas that are not well understood and on functional rather than non-functional requirements; error checking may be left out. **Benefits:** improved system usability, a closer match to users' real needs, improved design quality, improved maintainability and reduced development effort. Throw-away prototypes should be discarded, as they are undocumented, have a degraded structure and cannot be tuned to meet non-functional requirements.

(b) Incremental delivery. Rather than deliver the system as a single delivery, development and delivery are broken down into **increments**, each delivering part of the required functionality. User requirements are **prioritised** and the highest-priority ones go into early increments. Once an increment is started, **its requirements are frozen**; requirements for later increments can continue to evolve.

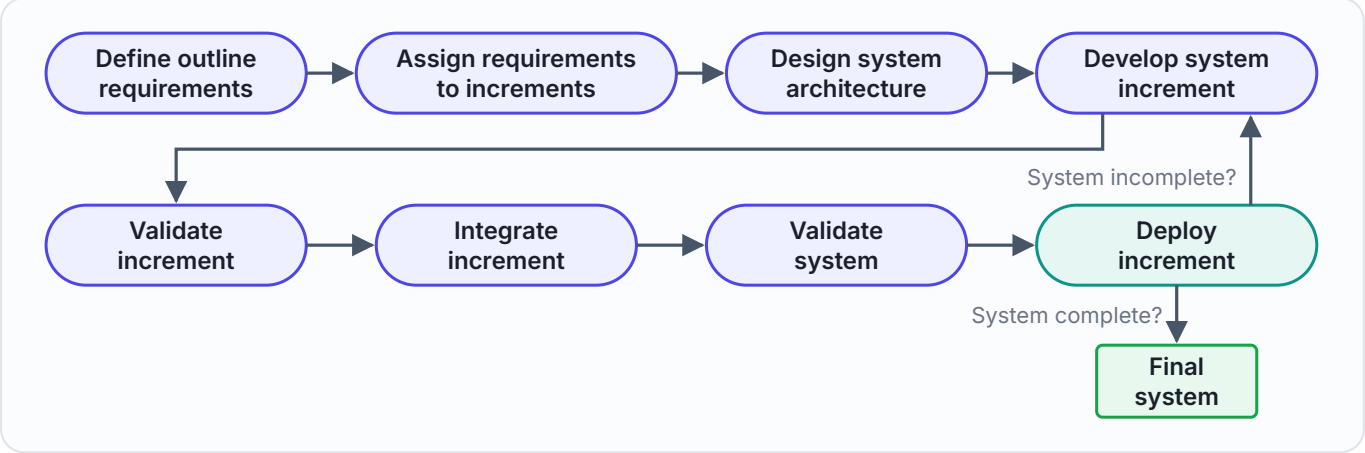


Fig Q5b Incremental delivery: the loop repeats once per increment. Sommerville Fig 2.10 · Study guide Fig 2.13

ADVANTAGES

- ✓ Customer value is delivered with each increment, so functionality is available earlier
- ✓ Early increments act as a prototype to help elicit requirements for later ones
- ✓ Lower risk of overall project failure
- ✓ The highest-priority services receive the most testing

PROBLEMS

- ✗ Common basic facilities needed by different parts are hard to identify early
- ✗ Conflicts with procurement models where the complete specification is part of the contract
- ✗ Hard to use for replacement systems: users won't accept less than the old system

Q6 Explain the process improvement cycle and the SEI capability maturity levels with a neat diagram.

5 MARKS | CO1 | L2 | LIKELY

Marks split → Definition 1 + Improvement cycle 1.5 + Five maturity levels with diagram 2.5

MODEL ANSWER

> **DEFINITION — LEARN THIS LINE**
> **Process improvement** means understanding existing processes and changing these processes to increase product quality and/or reduce costs and development time.

There are two approaches: the **process maturity approach** (improve process and project management and introduce good software engineering practice; the maturity level reflects how far good technical and management practice has been adopted) and the **agile approach** (iterative development and the reduction of overheads in the process).

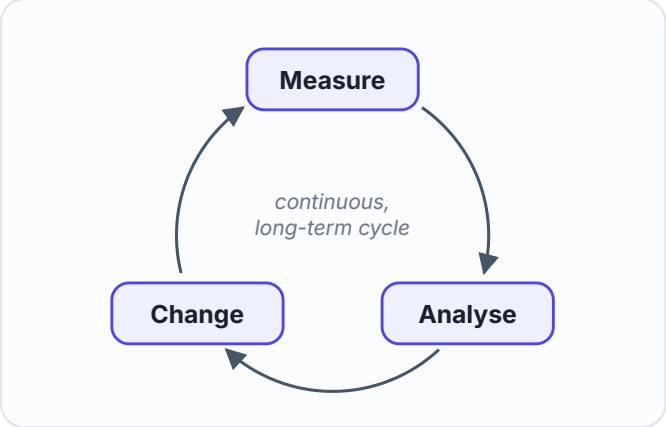


Fig Q6a The process improvement cycle. Sommerville Fig 2.11

1. **Process measurement**: measure one or more attributes of the process or product; these form a **baseline** to decide whether improvements have been effective.
2. **Process analysis**: assess the current process and identify **weaknesses and bottlenecks**; process models (process maps) may be developed.
3. **Process change**: propose and introduce changes for the weaknesses found; the cycle then resumes to collect data on their effectiveness.

Measurements (time taken, resources required, number of occurrences of an event such as defects found) should be used to **assess** improvements but should not **drive** them: the driver should be the **organisational objectives**.

SEI capability maturity levels. In the late 1980s the **Software Engineering Institute (SEI)** proposed a model of process capability maturity (Humphrey, 1988) so that the US Department of Defense could assess defence contractors. Its five levels correspond to the use of good software engineering practice (bottom to top):

1. **Initial**: essentially uncontrolled; process goals are satisfied and the scope of work is set out and communicated to the team.
2. **Repeatable** (CMMI: Managed): product management procedures defined and used; documented project plans, resource management and process monitoring.
3. **Defined**: process management procedures and strategies defined and used; standard organisational processes are adapted for each project.
4. **Managed** (CMMI: Quantitatively managed): quality management strategies defined and used; statistical methods control sub-processes using measurements.
5. **Optimising**: process improvement strategies defined and used; measurements and trends drive continuous improvement.

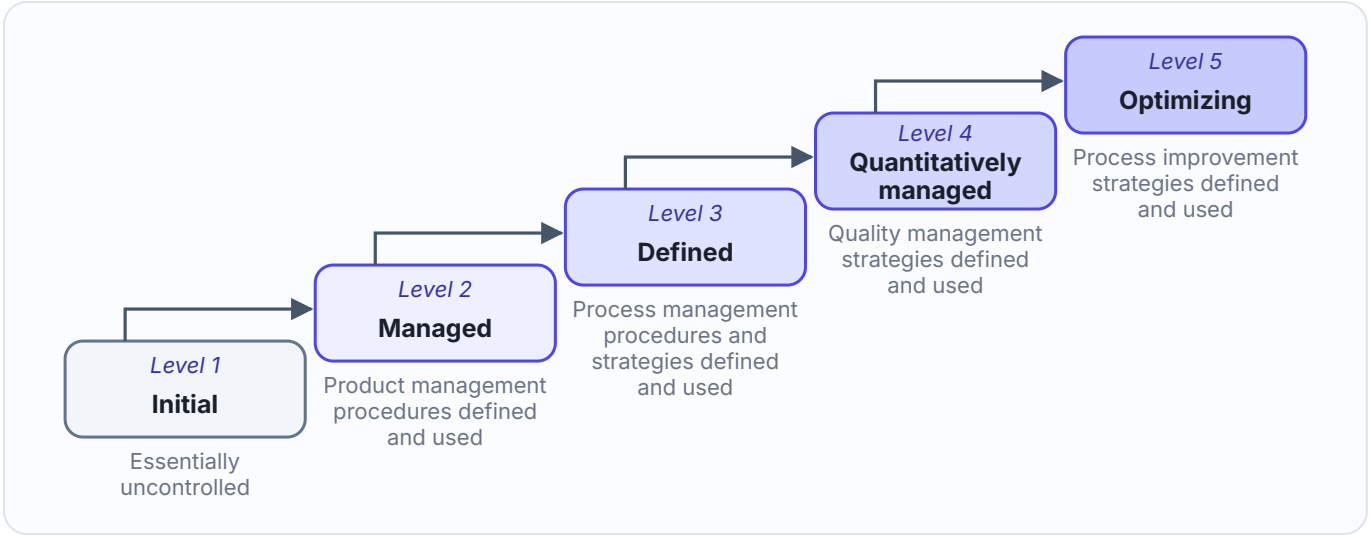


Fig Q6b Capability maturity levels (grey = what each level means). Sommerville Fig 2.12 · Study guide Fig 2.15

Limits: the model has too much overhead for small companies, and maturity estimation with agile processes is difficult, so it is used mainly by large companies.

MEMORY TRICK

SEI levels bottom to top: **I**nitial, **R**epeatable, **D**efined, **M**anaged, **O**ptimising: "**I R**eally **D**o **M**ean **O**rder". The cycle: **Measure → Analyse → Change**.

Q7 Describe and explain the advantages of the following XP practices: (i) refactoring (ii) pair programming (iii) user stories.

5 MARKS | CO1 | L1/L2 | ASKED IN CIE-1 2024 | VERY LIKELY

Marks split → XP intro 0.5 + Each practice: meaning and advantages 1.5 × 3

MODEL ANSWER

Extreme Programming (XP) is a very influential agile method that takes an "extreme" approach to iterative development: new versions may be built several times per day, increments are delivered to customers every 2 weeks, and all tests must be run for every build. Requirements are recorded as user stories, which drive each release:

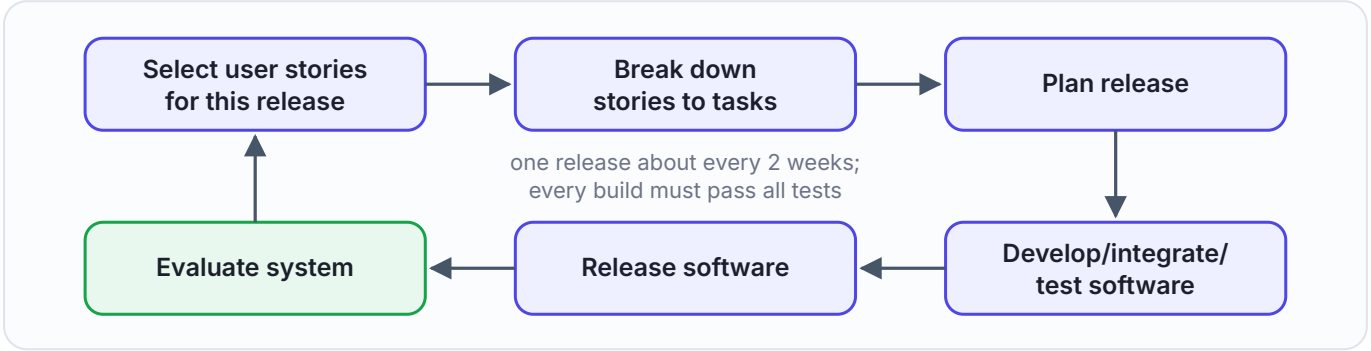


Fig Q7 The XP release cycle (optional sketch). Sommerville Fig 3.3 · Study guide Fig 3.3

(i) Refactoring

Refactoring is constant code improvement: the programming team look for possible improvements to the software and implement them immediately, even where there is no immediate need for them. XP argues that changes cannot be reliably anticipated, so instead of designing for change it keeps the code easy to

change. *Examples:* re-organising a class hierarchy to remove duplicate code; tidying up and renaming attributes and methods; replacing inline code with calls to methods in a program library.

- **Improves understandability** of the software, and so reduces the need for documentation.
- **Changes are easier to make**, because the code is well-structured and clear.
- **Stops the structure degrading** as increments are added, keeping the code simple and maintainable. (Limit: architecture refactoring is much more expensive.)

(ii) Pair programming

Programmers **work in pairs, sitting together at the same computer**, to develop the code. Pairs are created dynamically, so that all team members work with each other during development.

- **Common ownership of code**: knowledge is spread across the team, which shares responsibility for problems.
- **Informal review process**: each line of code is looked at by more than one person; cheaper and easier to organise than formal inspections.
- **Encourages refactoring**, because the whole team benefits from improving the code.
- **Reduces project risk** when team members leave. It is not necessarily inefficient: a pair can be as productive as two programmers working separately.

(iii) User stories

A **user story** is a scenario of use that might be experienced by a system user. The customer, who is part of the XP team, writes it on a **story card** with the developers. The team breaks each story into **implementation tasks**, which are the basis of schedule and cost estimates, and the customer chooses the stories for the next release by priority. *Example:* the Mentcare "Prescribing medication" story is split into tasks such as *Change dose of prescribed drug*, *Formulary selection* and *Dose checking*.

- People **relate to stories more easily** than to a requirements document or use cases.
- Stories **get users involved** in suggesting requirements.
- Tasks give a **direct basis for estimates and release planning**; unimplemented stories can change or be discarded as requirements change. (Problem: it is hard to judge whether the stories are complete.)

> ✎ **EXAM TIP**
> Structure each part as **meaning (2 lines) + 3 advantages**. Name one refactoring example and the story-to-task idea; that is what examiners look for.

Q8 List the principles of agile methods and explain the Scrum sprint cycle with a neat labelled diagram.

5 MARKS | CO1 | L2 | ASKED IN CIE-1 2024 | VERY LIKELY

Marks split → Five principles 2 + Sprint cycle diagram 1.5 + Explanation 1.5

MODEL ANSWER

Agile methods are incremental methods focused on rapid development, frequent releases and minimal documentation. The **Agile Manifesto** values *individuals and interactions* over processes and tools, *working software* over comprehensive documentation, *customer collaboration* over contract negotiation, and *responding to change* over following a plan. All agile methods share five principles:

Principle	Description
1. Customer involvement	Customers are closely involved throughout development; they provide and prioritise new requirements and evaluate the iterations.
2. Incremental delivery	The software is developed in increments, with the customer specifying the requirements to be included in each increment.
3. People, not process	The skills of the team are recognised and exploited; members develop their own ways of working without prescriptive processes.
4. Embrace change	Expect the system requirements to change, and so design the system to accommodate these changes.
5. Maintain simplicity	Focus on simplicity in the software and in the process; actively work to eliminate complexity.

Scrum is an agile method that focuses on **managing iterative development**. Its framework is centred on a set of **sprints**: fixed time periods (usually 2–4 weeks) in which a system increment is developed. A Scrum project has an outline planning phase, a series of sprint cycles, and a project closure phase.

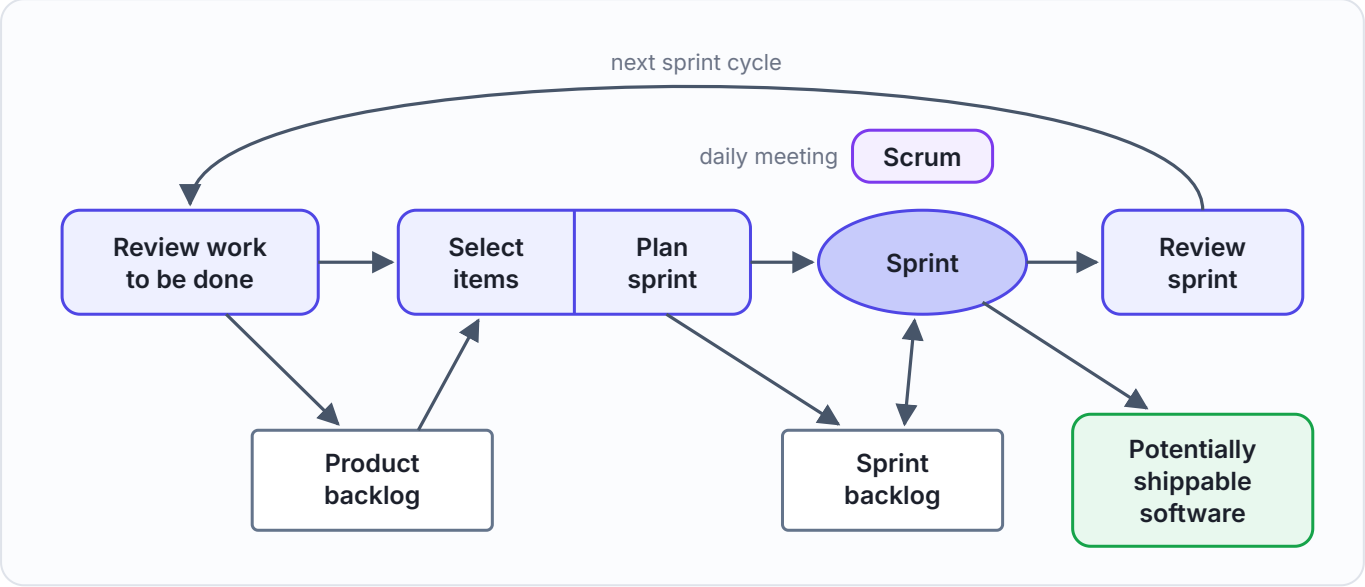


Fig Q8 The Scrum sprint cycle. Sommerville Fig 3.9 · Study guide Fig 3.9

1. **Product backlog**: the starting point; the list of work to be done (features, requirements, user stories, engineering tasks), prioritised by the **Product Owner**.

2. **Review work and select items**: the whole team works with the customer to select the features and functionality to be developed during the sprint.
3. **Plan sprint**: the team estimates the work using its **velocity** and produces the **sprint backlog**; the team then organises itself to develop the software.
4. **Sprint**: fixed length (2–4 weeks); the team is isolated, and all outside communication goes through the **ScrumMaster**, who protects it from distractions. A short daily **Scrum** meeting reviews progress.
5. **Review sprint**: the work done is reviewed and presented to stakeholders; the output is **potentially shippable software**, and the next sprint cycle begins.

> **MEMORY TRICK**
>
> Agile principles: **"Can I Pass Every Module?"** = **C**ustomer involvement, **I**ncremental delivery, **P**eople not process, **E**mbrace change, **M**aintain simplicity.

Backup questions (prepare if you have time)

5M **Explain the integration and configuration (reuse-oriented) model with a neat diagram. Give its advantages and disadvantages.**

Answer outline → System assembled from existing configurable components or application systems (COTS). Reusable software: stand-alone application systems, collections of objects (.NET/J2EE), web services. Stages: requirements specification → software discovery and evaluation → requirements refinement → application system configuration, or component adaptation and integration. Pros: reduced cost and risk, faster delivery. Cons: requirements compromises, loss of control over evolution. Draw Fig 2.3 (study guide 2.2).

5M **Explain Boehm's spiral model with a neat diagram.**

Answer outline → Boehm (1988): evolutionary, risk-driven; couples the iteration of prototyping with the control of waterfall. Clockwise from the centre through communication, planning (with risk analysis), modeling, construction, deployment; each circuit gives a more complete version; risk considered on every revolution; anchor point milestones. 2 advantages + 2 disadvantages. Draw Fig 2.6 (study guide 2.3).

5M **Explain the four fundamental software process activities. Draw the V-model showing the testing phases in a plan-driven process.**

Answer outline → Specification, design and implementation, validation, evolution (one line each). Testing stages: component → system → customer (acceptance). V-model test plans: acceptance, system integration, sub-system integration; detailed design → module and unit code and test; tests run back up to service. Draw Fig 2.10 (study guide 2.1, 2.4).

5M **Differentiate between plan-driven and agile development.**

Answer outline → Plan-driven: separate stages, outputs planned in advance, iteration *within* activities. Agile: activities interleaved, outputs negotiated, iteration *across* activities. Table rows: communication, delivery, change, best suited for. Most real processes mix both. Draw Fig 3.1 (study guide 3.1).

5M **Explain test-first development in XP and the Scrum terminology.**

Answer outline → Test-first: tests written before the code clarify the requirements; tests are programs (JUnit) and all old and new tests run automatically on each build; 3 problems (short cuts, hard-to-write tests, completeness). Scrum terms: development team (≤ 7), product backlog, product owner, Scrum, ScrumMaster, sprint, velocity, potentially shippable increment (study guide 3.6, 3.8).

UNIT 2 · MOST LIKELY 5-MARK QUESTIONS

Unit 2: 8 questions you must prepare

CO-2 · Functional & non-functional requirements · RE process · Elicitation · Requirements document · Validation · Design concepts · Design model

QUESTIONS	FROM	TIME PER ANSWER
8 model answers + 5 backups	Study guide Chapters 4 and 5	About 10 minutes in the exam

QUESTIONS AT A GLANCE

1. Functional vs non-functional requirements
2. NFRs: types and metrics *(asked 2024)*
3. Spiral RE process *(asked 2024)*
4. Elicitation & analysis; interviews, ethnography
5. Structure of the requirements document *(asked 2024)*
6. Requirements validation: checks & techniques
7. Abstraction, modularity, hiding, independence
8. Requirements model → design model; its elements

Q1 Differentiate between functional and non-functional requirements with examples.

5 MARKS · CO2 · L2 · VERY LIKELY

Marks split → Definitions 1 + Comparison table 2 + Examples 2

MODEL ANSWER

Functional requirements (FRs) are statements of services the system should provide, how the system should react to particular inputs and how the system should behave in particular situations. They may also state what the system should *not* do.

Non-functional requirements (NFRs) are constraints on the services or functions offered by the system, such as timing constraints, constraints on the development process and standards. They often apply to the system as a whole rather than to individual features or services.

Basis	Functional requirements	Non-functional requirements
Meaning	Services the system provides; its reactions to inputs and behaviour in particular situations	Constraints on those services; properties such as reliability, response time, storage
Answers	**What** should the system do?	**How well**, and under what constraints, must it do it?
Scope	Usually one feature or service	Often the whole system (emergent properties)
Types	Functional user requirements and functional system requirements	Product, organisational and external requirements

Basis	Functional requirements	Non-functional requirements
If not met	Users can often work around the gap	System may be useless (e.g. an aircraft system not certified as safe)
Verified by	Testing that the function gives the right output for given inputs	Measuring with metrics: response time, mean time to failure, training time

Examples

System	Functional requirement	Non-functional requirement
Mentcare (patient information system)	1. A user shall be able to search the appointments lists for all clinics. 2. The system shall generate each day, for each clinic, a list of patients who are expected to attend appointments that day.	1. The system shall be available to all clinics during normal working hours (Mon–Fri, 08:30–17:30); downtime shall not exceed five seconds in any one day. *(product)* 2. Users shall authenticate themselves using their health authority identity card. *(organisational)*
Insulin pump	The system shall measure the blood sugar and deliver insulin, if required, every 10 minutes.	The system shall be available to deliver insulin when required. *(product → dependability)*

- **The line is not sharp:** a security NFR (only authorised users may access the system) generates FRs such as user authentication.
- **Two levels:** both are written as **user requirements** (natural language + diagrams, for customers) and **system requirements** (detailed and structured, for developers; may be part of the contract).
- **Imprecision:** FRs must be precise — "search" may mean all clinics (user) or one clinic (developer's simpler reading), causing delays and cost.

> ✎ **EXAM TIP**
> Give one FR and one NFR from the **same** system, and label each NFR with its type (product, organisational or external). That shows the examiner you know the classification too.

Q2 What are non-functional requirements? List the types of non-functional requirements. What are the metrics for specifying non-functional requirements?

5 MARKS · CO2 · L2 · ASKED IN CIE-1 2024 · VERY LIKELY

Marks split → Definition 1 + Types with diagram 2 + Metrics 2

MODEL ANSWER

Non-functional requirements are constraints on the services or functions offered by the system, such as timing constraints, constraints on the development process and standards. They define system properties and constraints (e.g. reliability, response time, storage requirements) and often apply to the system as a whole. They may be **more critical than functional requirements**: if they are not met, the system may be useless.

Types of NFRs (classified by where they come from):

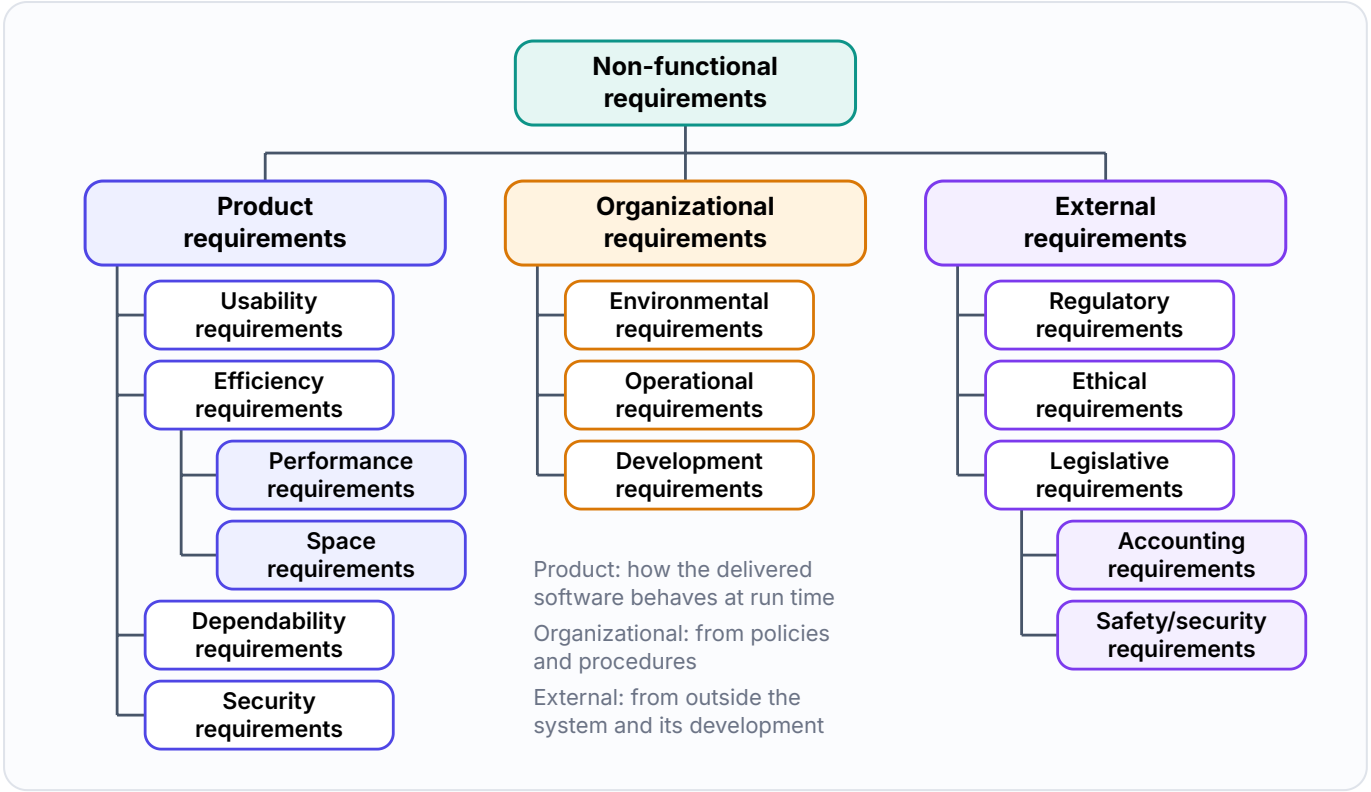


Fig Q2 Types of non-functional requirement. Shaded boxes are sub-types (Efficiency → Performance, Space; Legislative → Accounting, Safety/security). Sommerville Fig 4.3 · study guide Fig 4.5

1. **Product requirements** — specify that the delivered product must behave in a particular way (run-time behaviour): usability, efficiency (performance, space), dependability, security. *E.g. Mentcare shall be available to all clinics Mon–Fri, 08:30–17:30.*
2. **Organisational requirements** — a consequence of organisational policies and procedures: environmental, operational, development (programming language, development environment, process standards). *E.g. users shall authenticate with their health authority identity card.*
3. **External requirements** — arise from factors external to the system and its development process: regulatory, ethical, legislative (accounting, safety/security). *E.g. the system shall implement the patient privacy provisions in HStan-03-2006-priv.*

Metrics. A vague **goal** ("easy to use") cannot be tested, so NFRs should be written as **verifiable requirements** — statements using some measure that can be objectively tested. The metrics are:

Property	Measure
Speed	Processed transactions/second · User/event response time · Screen refresh time
Size	Mbytes · Number of ROM chips
Ease of use	Training time · Number of help frames
Reliability	Mean time to failure · Probability of unavailability · Rate of failure occurrence · Availability
Robustness	Time to restart after failure · Percentage of events causing failure · Probability of data corruption on failure
Portability	Percentage of target dependent statements · Number of target systems

Example: the goal "the system should be easy to use by medical staff" becomes the testable NFR "Medical staff shall be able to use all the system functions after four hours of training; after this, the average number of errors made by experienced users shall not exceed two per hour of system use."

MEMORY TRICK

Types — **P-O-E**: Product → *U Eat Dosa Slowly* (Usability, Efficiency, Dependability, Security) · Organisational → *Every Office Decides* (Environmental, Operational, Development) · External → the *REaL* world (Regulatory, Ethical, Legislative).
Metrics — "**S**ix **S**mart **E**ngineers **R**arely **R**un **P**oorly": Speed, Size, Ease of use, Reliability, Robustness, Portability.

Q3 Explain the requirements engineering process with a neat labelled spiral diagram.

5 MARKS | CO2 | L2 | ASKED IN CIE-1 2024 | VERY LIKELY

Marks split → Activities 1 + Spiral diagram 2 + Explanation 2

MODEL ANSWER

Requirements engineering (RE) is the process of establishing the services that a customer requires from a system and the constraints under which it operates and is developed. It has three key activities — **requirements elicitation and analysis** (discovering requirements by interacting with stakeholders), **requirements specification** (converting them into a standard form) and **requirements validation** (checking that they define the system the customer wants) — plus **requirements management** to handle change. In practice these activities are interleaved and organised as an iterative process around a **spiral**.

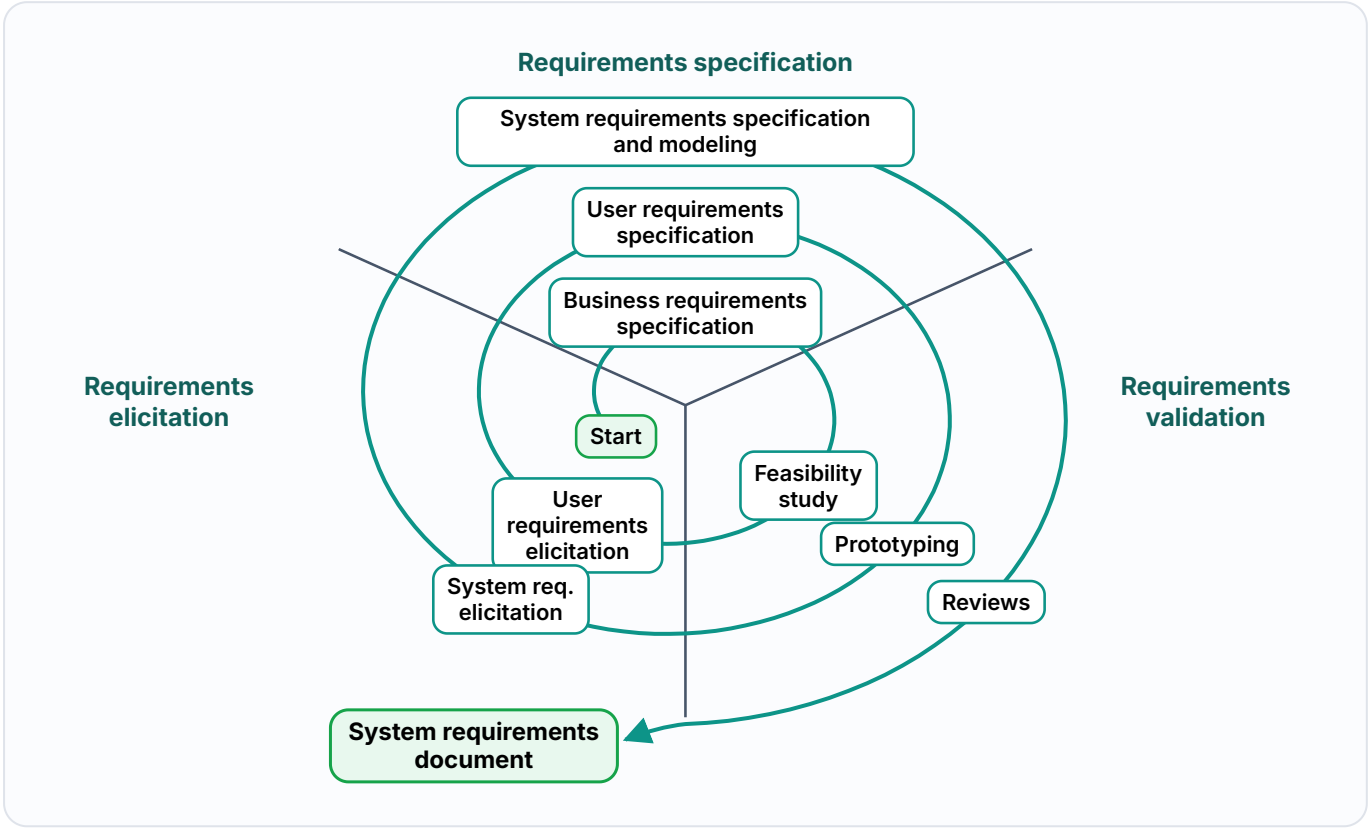


Fig Q3 A spiral view of the requirements engineering process. Sommerville Fig 4.6 · study guide Fig 4.7

1. **Three sectors:** every loop of the spiral passes through elicitation → specification → validation.

2. **More detail each loop:** inner loop — business requirements specification and feasibility study; middle loop — user requirements elicitation and specification, checked by prototyping; outer loop — system requirements elicitation, specification and modeling, checked by reviews.
3. **Effort varies:** the time spent on each activity depends on the stage of the process, the type of system and the budget. Early on, effort goes on high-level business and non-functional requirements and user requirements; later, on detailed system requirements.
4. **Number of loops can vary:** the spiral can be exited after some or all user requirements are elicited; agile development can be used instead of prototyping, so requirements and implementation are developed together.
5. **Output:** the **system requirements document**.
6. **Change:** in virtually all systems requirements change, so requirements management runs alongside the spiral.

> ✎ **EXAM TIP**
> Draw a "Y" (three lines from a centre); label **specification** (top), **elicitation** (left), **validation** (right); draw the spiral outwards from *Start* and write the nine activities in order — business req. spec → feasibility study → user req. elicitation → user req. spec → prototyping → system req. elicitation → system req. spec & modeling → reviews — ending in the system requirements document.

Q4 Explain the requirements elicitation and analysis process with a neat diagram. Briefly explain interviewing and ethnography.

5 MARKS | CO2 | L2 | VERY LIKELY

Marks split → Diagram 1.5 + Four activities 1.5 + Interviewing 1 + Ethnography 1

MODEL ANSWER

Requirements elicitation and analysis involves technical staff working with customers to find out about the application domain, the services that the system should provide and the system's operational constraints. It may involve end-users, managers, engineers involved in maintenance, domain experts, trade unions, etc. — the **stakeholders**.

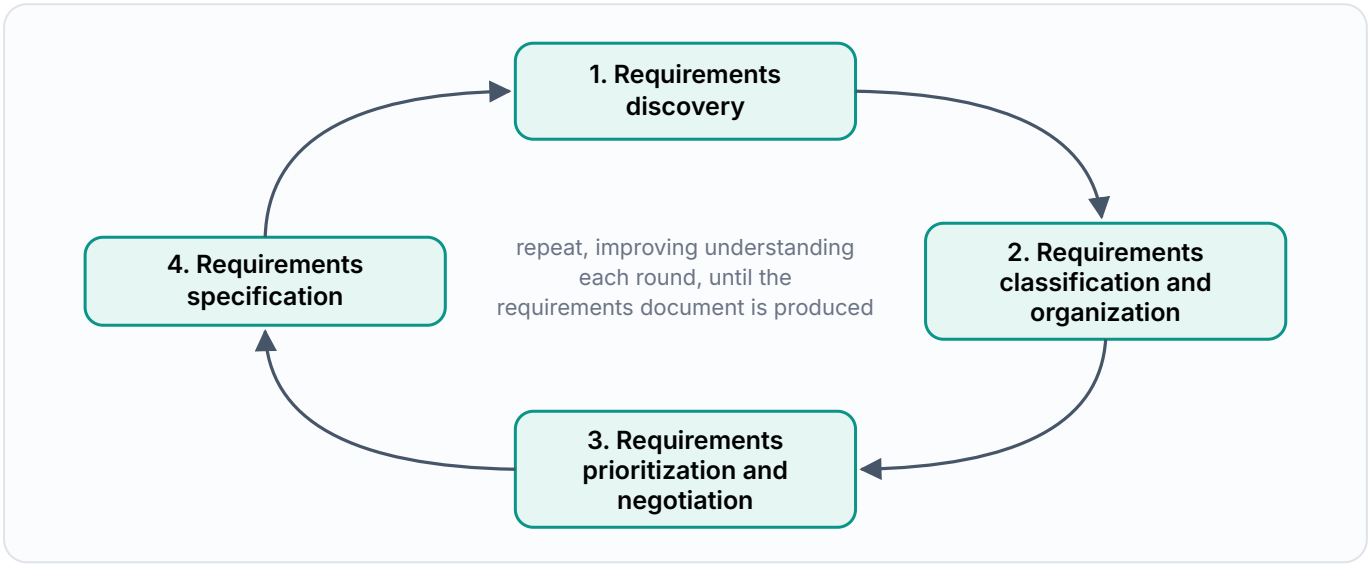


Fig Q4 The requirements elicitation and analysis process. Sommerville Fig 4.7 · study guide Fig 4.8

1. **Requirements discovery** — interacting with stakeholders to discover their requirements; domain requirements are also discovered here.
2. **Requirements classification and organisation** — takes the unstructured collection of requirements, groups related requirements and organises them into coherent clusters.
3. **Prioritisation and negotiation** — prioritising requirements and resolving requirements conflicts through negotiation between stakeholders.
4. **Requirements specification** — requirements are documented and input into the next round of the spiral.

The process is **iterative**, with continual feedback between activities; the analyst's understanding improves with each round.

Interviewing

Formal or informal interviews in which the RE team asks stakeholders about the system they use now and the system to be developed.

- **Closed interviews** — based on a pre-determined list of questions.
- **Open interviews** — no predefined agenda; various issues are explored with stakeholders.

In practice a mix of both. The interviewer must be open-minded and prompt discussion with a springboard question, a requirements proposal or a prototype. Weak for domain requirements (jargon; knowledge too familiar to articulate).

Ethnography

An **observational technique**: a social scientist spends a considerable time observing and analysing how people actually work. People do not have to explain their work, and social and organisational factors can be observed. It finds requirements from how people **actually work** and from **cooperation and awareness** of others' activities, but cannot identify new features.

Focused ethnography (developed in an air traffic control project) combines ethnography with **prototyping**: prototype development raises unanswered questions that focus the ethnographic analysis.

MEMORY TRICK

The four activities in order — **D-C-P-S**: "*Doctors Collect, Prioritise, Sign*" (Discovery, Classification & organisation, Prioritisation & negotiation, Specification).

Q5 Explain the structure of the IEEE software requirements specification (requirements document).

5 MARKS | CO2 | L2 | ASKED IN CIE-1 2024 | VERY LIKELY

Marks split → Definition 1 + Chapter structure 3 + Users & notes 1

MODEL ANSWER

The **software requirements document** (software requirements specification, **SRS**) is the official statement of what is required of the system developers. It includes both a definition of user requirements and a specification of the system requirements. It is **not a design document**: it sets out **what** the system should do rather than **how**. One possible structure, based on the IEEE standard, is:

#	Chapter	Description
1	**Preface**	Defines the expected readership; version history, with a rationale for each new version and a summary of changes.
2	**Introduction**	Describes the need for the system, briefly its functions and how it works with other systems, and how it fits the organisation's business or strategic objectives.
3	**Glossary**	Defines the technical terms used; makes no assumptions about the reader's experience or expertise.
4	**User requirements definition**	Services provided for the user, and the non-functional requirements, in natural language and diagrams customers understand; product and process standards to be followed.
5	**System architecture**	High-level overview of the anticipated architecture, showing the distribution of functions across system modules; reused components are highlighted.
6	**System requirements specification**	Functional and non-functional requirements in more detail; interfaces to other systems may be defined.
7	**System models**	Graphical models showing relationships between system components and between the system and its environment (object, data-flow, semantic data models).
8	**System evolution**	Fundamental assumptions on which the system is based, and anticipated changes due to hardware evolution, changing user needs, etc.
9	**Appendices**	Detailed application-specific information, e.g. hardware requirements (minimal and optimal configurations) and database requirements (logical organisation of data).
10	**Index**	A normal alphabetic index, plus indexes of diagrams, functions, and so on.

Users of the document — it is organised so that both customers and developers can use it:

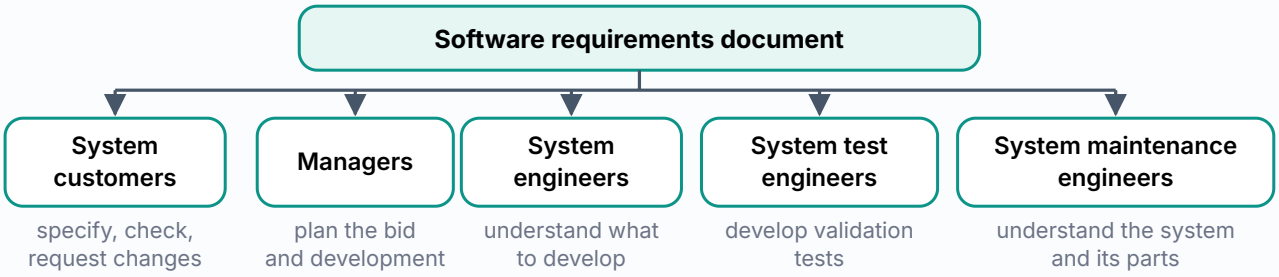


Fig Q5 Users of a requirements document and how they use it. Sommerville Fig 4.16 · study guide Fig 4.11

Variability: the information included depends on the type of system and the approach to development. Incrementally developed systems have less detail; critical and outsourced systems need detailed, precise specifications. The IEEE standard is mostly applicable to large systems engineering projects.

> **MEMORY TRICK**
>
> **"PIG-U, four S's, A-I"**: **P**reface, **I**ntroduction, **G**lossary, **U**ser requirements definition → **S**ystem architecture, **S**ystem requirements specification, **S**ystem models, **S**ystem evolution → **A**ppendices, **I**ndex.

Q6 What is requirements validation? Explain the requirements checks and validation techniques.

5 MARKS | CO2 | L2 | LIKELY

Marks split → Definition 1 + Five checks 2 + Techniques & reviews 2

MODEL ANSWER

Requirements validation is the process of demonstrating that the requirements define the system that the customer really wants. It overlaps with elicitation and analysis, as it is concerned with finding problems with the requirements. It is important because requirements error costs are high: fixing a requirements error after delivery may cost up to **100 times** the cost of fixing an implementation error, since the design and implementation must also change and the system must be retested.

Requirements checks

Check	Question asked
1. Validity	Does the system provide the functions which best support the customer's needs?
2. Consistency	Are there any requirements conflicts (contradictory constraints, different descriptions of the same function)?
3. Completeness	Are all functions required by the customer included?
4. Realism	Can the requirements be implemented given the available budget and technology?
5. Verifiability	Can the requirements be checked (can tests show the delivered system meets each one)?

Requirements validation techniques

1. **Requirements reviews** — systematic manual analysis of the requirements by a team of reviewers, who check for errors and inconsistencies.
2. **Prototyping** — using an executable model of the system to check requirements; end-users and customers experiment with it and feed back requirements changes.
3. **Test-case generation** — developing tests for requirements to check testability. If a test is difficult or impossible to design, the requirement will be hard to implement and should be reconsidered.

Requirements reviews

Regular reviews should be held while the requirements definition is being formulated. Both client and contractor staff should be involved. Reviews may be formal (with completed documents) or informal; good communication between developers, customers and users can resolve problems at an early stage. **Review checks:**

Review check	Question
Verifiability	Is the requirement realistically testable?
Comprehensibility	Is the requirement properly understood?
Traceability	Is the origin of the requirement clearly stated?
Adaptability	Can the requirement be changed without a large impact on other requirements?

MEMORY TRICK

Checks — **V-C-C-R-V**: "*Very Careful Clients Request Verification*" (Validity, Consistency, Completeness, Realism, Verifiability). Techniques — **R-P-T**: Reviews, Prototyping, Test cases. Do not mix the 5 checks with the 4 review checks; only verifiability is in both.

Q7 Explain the following design concepts: abstraction, modularity, information hiding and functional independence (cohesion and coupling).

5 MARKS | CO2 | L2 | VERY LIKELY

Marks split → Abstraction 1 + Modularity with graph 1.5 + Information hiding 1 + Functional independence 1.5

MODEL ANSWER

1. Abstraction. A modular solution can be stated at many levels of abstraction: at the **highest** level in broad terms using the language of the problem environment; at **lower** levels in more detail, using implementation-oriented terms; at the **lowest** level in a form that can be directly implemented.

- **Procedural abstraction** — a sequence of instructions that have a specific and limited function; the name implies the function but specific details are suppressed. *E.g. "open" a door (walk to the door, grasp the knob, turn and pull...).*
- **Data abstraction** — a named collection of data that describes a data object. *E.g. "door": door type, swing direction, opening mechanism, weight, dimensions.*

2. Modularity. Software is divided into **separately named and addressable components**, called modules, that are integrated to satisfy problem requirements. "Modularity is the single attribute of software that allows a program to be intellectually manageable" (Myers). But modules should not be split indefinitely:

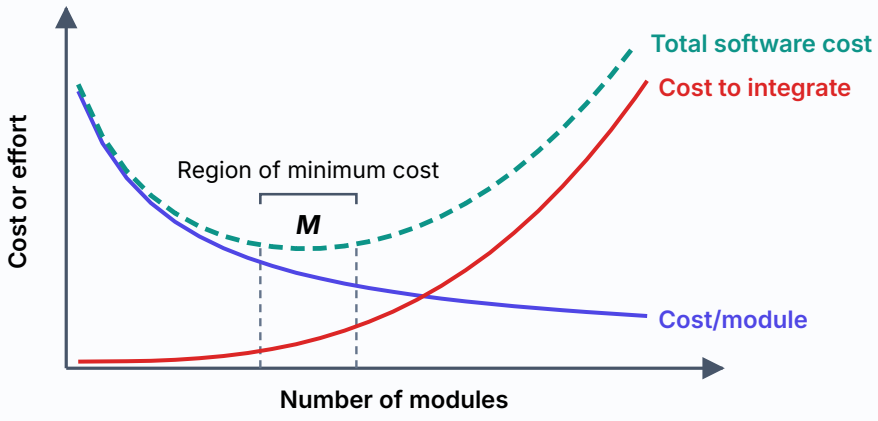


Fig Q7 Modularity and software cost: total cost is lowest in the region near *M* modules. Pressman Fig 8.2 · study guide Fig 5.4

As the number of modules grows, the **cost per module falls** but the **cost to integrate rises**. Total cost is lowest near some number *M*, so we modularize but stay near *M* — **avoid under-modularity and over-modularity**.

3. Information hiding (Parnas). Modules should be "characterized by design decisions that (each) hides from all others": the **information (algorithms and data) inside a module is inaccessible to other modules that have no need for it**. Modules communicate only the information necessary, through an interface. **Benefit:** during testing and maintenance, errors introduced by a modification are less likely to propagate to other parts of the software.

4. Functional independence is achieved by developing modules with a **"single-minded" function** and an **"aversion" to excessive interaction** with other modules: each module addresses a specific subset of requirements and has a simple interface. Such modules are easier to develop, test and maintain, and errors propagate less. It is assessed by two criteria:

- **Cohesion** — an indication of the relative functional strength of a module. A cohesive module performs a single task and ideally does just one thing. Aim: **high cohesion**.
- **Coupling** — an indication of the relative interdependence among modules; it depends on interface complexity, the point at which entry or reference is made, and what data pass across the interface. Aim: **low coupling**, which avoids the "ripple effect" of errors.

MEMORY TRICK

Co-HE-sion = inside one module; COUPLE-ing = between two. Good design = **high cohesion, low coupling** — "strong teams, few meetings".

Q8 Explain how the requirements model is translated into the design model, and describe the elements of the design model.

5 MARKS | CO2 | L2 | LIKELY

Marks split → Translation diagram 2 + Explanation 1 + Elements 2

MODEL ANSWER

Design is the last software engineering action within the modeling activity and sets the stage for construction. The **requirements model** — its scenario-based, class-based, flow-oriented and behavioural elements — feeds the design task. Using design notation and methods, design produces the **design model**: a representation of the software that gives detail about the architecture, data structures, interfaces and components needed to implement the system.

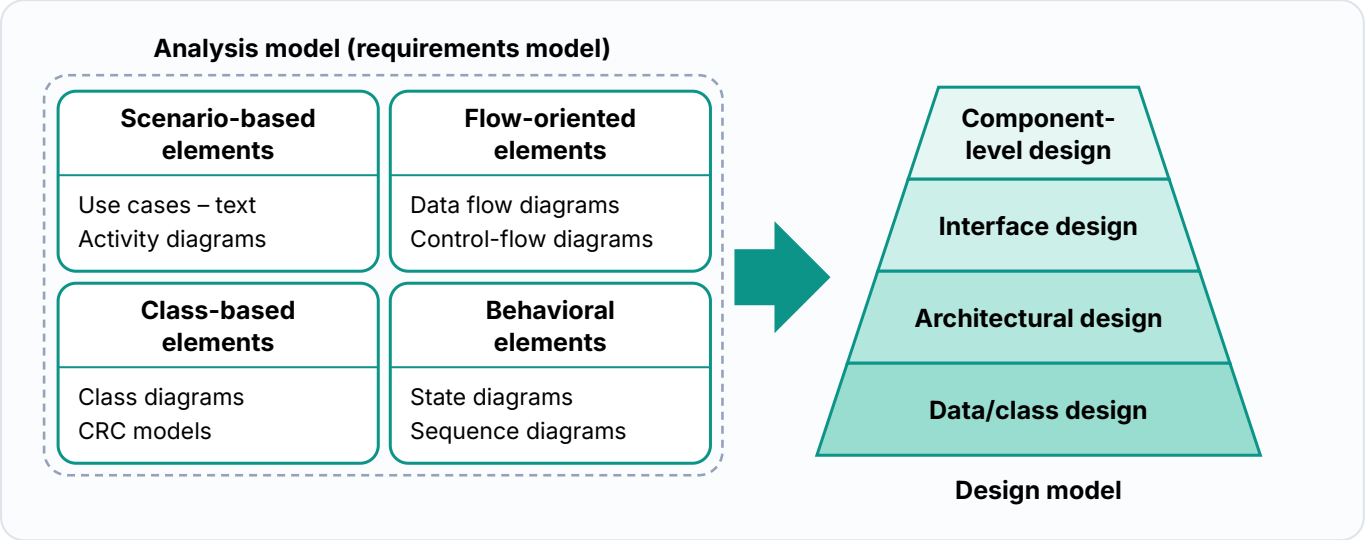


Fig Q8 Translating the requirements model into the design model. Pressman Fig 8.1 (simplified) · study guide Fig 5.2

- **Data/class design** — transforms class models into design class realizations and the data structures needed for implementation (from CRC objects, class attributes).
- **Architectural design** — defines the relationships between major structural elements, the architectural styles and design patterns, and the constraints on implementation.

- **Interface design** — how the software communicates with interoperating systems and with the humans who use it (from usage scenarios and behavioural models).
- **Component-level design** — transforms structural elements of the architecture into a procedural description of software components.

Design should **begin with data**, the foundation (base of the pyramid); then the architecture is derived. The design model has two dimensions: the **process dimension** (how the model evolves as design tasks are done) and the **abstraction dimension** (level of detail).

Elements of the design model

1. **Data design elements** — (data architecting) a high-level model of data refined into implementation-specific representations: data structures (component level), a database (application level), a data warehouse (business level).
2. **Architectural design elements** — like the **floor plan** of a house: an overall view of the software as a set of interconnected subsystems, derived from the application domain, requirements model elements and available architectural styles and patterns.
3. **Interface design elements** — like the doors, windows and external utilities: how information flows into and out of the system and among its components; three parts — user interface, external interfaces, internal interfaces.
4. **Component-level design elements** — like the detailed drawing of each room: the internal detail of each component — data structures, algorithmic detail and an interface to its operations (UML component and activity diagrams, pseudocode).
5. **Deployment-level design elements** — how software functionality and subsystems are allocated within the physical computing environment (UML deployment diagram, first in descriptor form, later in instance form).

> ✎ **EXAM TIP**
>
> Write all **five** elements (the textbook counts four in some places). Draw the pyramid with **data/class design at the base** — it is the foundation for everything above it.

Backup questions (prepare if you have time)

5M **Explain the requirements change management process with a diagram. Differentiate between enduring and volatile requirements.**

Answer outline → Why requirements change (business/technical environment changes, payers ≠ users, diverse user community) → draw: identified problem → **problem analysis and change specification** → **change analysis and costing** → **change implementation** → revised requirements, one line per stage → **enduring** = core, slow-to-change activities (recording patients' treatments) vs **volatile** = supporting activities likely to change (report layout, login method). Revise 4.7, Fig 4.13.

5M **Explain the ways of writing a system requirements specification. Explain structured specifications and use cases.**

Answer outline → Five notations (N-S-D-G-M): natural language, structured natural language, design description languages, graphical notations, mathematical specifications → structured spec = standard form; fields: function, inputs + source, outputs + destination, requires, action, pre/post-condition, side effects (insulin pump dose form) → use cases: UML scenarios; actors as stick figures, interactions as named ellipses; draw Mentcare use cases. Revise 4.5, Fig 4.10.

5M **List the design quality guidelines. Explain the FURPS quality attributes.**

Answer outline → Eight guidelines: architecture (recognisable styles/patterns, good components, evolutionary), modular, distinct representations of data, architecture, interfaces, components, appropriate data structures, independent components, interfaces that reduce complexity, repeatable method, notation that communicates → FURPS table: Functionality, Usability, Reliability, Performance, Supportability with what each is assessed by. Revise 5.2.

5M **Differentiate between cohesion and coupling. Why are high cohesion and low coupling desired?**

Answer outline → Both definitions (functional strength vs interdependence) → table: measures inside vs between modules; goal high vs low; depends on related functions vs interface complexity, entry point, data passed; bad case "schizophrenic" module vs ripple effect → draw good vs poor modules (Fig 5.6) → benefits: easier to develop, test, maintain, reuse. Add the cohesion and coupling types only if asked. Revise 5.3 (concept 7).

5M **Who are system stakeholders? Differentiate between user requirements and system requirements.**

Answer outline → Stakeholder = any person or organisation affected by the system with a legitimate interest → types: end users, system managers, system owners, external stakeholders; Mentcare examples (patients, doctors, nurses, receptionists, IT staff) → user vs system table (detail, notation, readers, purpose) → Mentcare drug-cost example (1 → 1.1–1.5) → readers diagram. Revise 4.1, Figs 4.1–4.2.

UNIT 3 · MOST LIKELY 5-MARK QUESTIONS

Unit 3: 7 questions you must prepare

CO-3 · Architectural design · Design decisions · Non-functional requirements · 4+1 views · Box-and-line models · Packing robot. Q1 is compulsory with one part per unit, so a Unit 3 question is effectively guaranteed.

QUESTIONS	FROM	TIME PER ANSWER
7 model answers + 4 backups	Study guide Chapter 6 (6.1–6.10)	About 10 minutes in the exam

QUESTIONS AT A GLANCE

1. What is architectural design? Useful views (asked in 2024)
2. Krutchen's 4+1 view model with a diagram
3. Architectural design decisions: the 8 fundamental questions
4. How non-functional requirements shape the architecture
5. Advantages of explicit architecture; architecture in the small vs large
6. Box-and-line diagrams and the two uses of architectural models
7. Packing robot example; architecture in agile; architecture reuse

Q1 What is software architecture design? What views or perspectives are useful when designing and documenting a system's architecture?

5 MARKS · CO3 · L2 · ASKED IN CIE-1 2024 · VERY LIKELY

Marks split → Definition + 3 points 2 + Diagram 1 + Views 2

MODEL ANSWER

DEFINITION — LEARN THIS LINE

Architectural design — is concerned with understanding how a software system should be organized and designing the overall structure of that system.

1. **Overall organization** — the software architecture shows the main components (sub-systems) of a system and the relationships between them.
2. **Critical link** — it is the first stage of the software design process and the critical link between design and requirements engineering, because it identifies the main structural components in a system and the relationships between them.
3. **Output** — an **architectural model** that describes how the system is organized as a set of communicating components. The architecture is the dominant influence on non-functional characteristics such as performance, robustness, distributability and maintainability.

Views. Each architectural model shows only **one view or perspective** of the system: how it is decomposed into modules, how the run-time processes interact, or how components are distributed across a network. So, for both design and documentation, we present **multiple views**. Krutchen's **4+1 view model** gives four fundamental views, linked by use cases or scenarios:



Fig Q1 The 4+1 view model, simplified for a quick sketch in the exam. Simplified from Sommerville Fig 6.3

1. **Logical view** — the key abstractions in the system as objects or object classes; requirements can be related to its entities.
2. **Process view** — how, at run-time, the system is composed of interacting processes; useful for judging non-functional characteristics such as performance and availability.
3. **Development view** — how the software is decomposed for development into components implemented by a single developer or team; useful for software managers and programmers.
4. **Physical view** — the system hardware and how software components are distributed across the processors; useful for systems engineers planning a deployment.

+1 Use cases or scenarios relate the four views to each other. Hofmeister et al. add a **conceptual view** (an abstract view, e.g. the packing robot diagram). In practice it is rarely necessary to describe all perspectives: develop the views that are useful for communication.

> ✎ **EXAM TIP**
> Asked as Q1(c) in 2024. Both halves carry marks: definition + 3 points first, then the 4+1 sketch and one line per view. Use the exact view names: logical, process, development, physical, +1 scenarios.

Q2 Explain Krutchen's 4+1 view model of software architecture with a neat diagram.

5 MARKS | CO3 | L2 | VERY LIKELY

Marks split → Diagram 2 + Four views 2 + Scenarios (+1) 1

MODEL ANSWER

No single diagram can show everything about a system: **each architectural model shows only one view or perspective**. In his well-known **4+1 view model** of software architecture, Krutchen suggests **four fundamental architectural views**, which are related using common **use cases or scenarios** (the "+1").

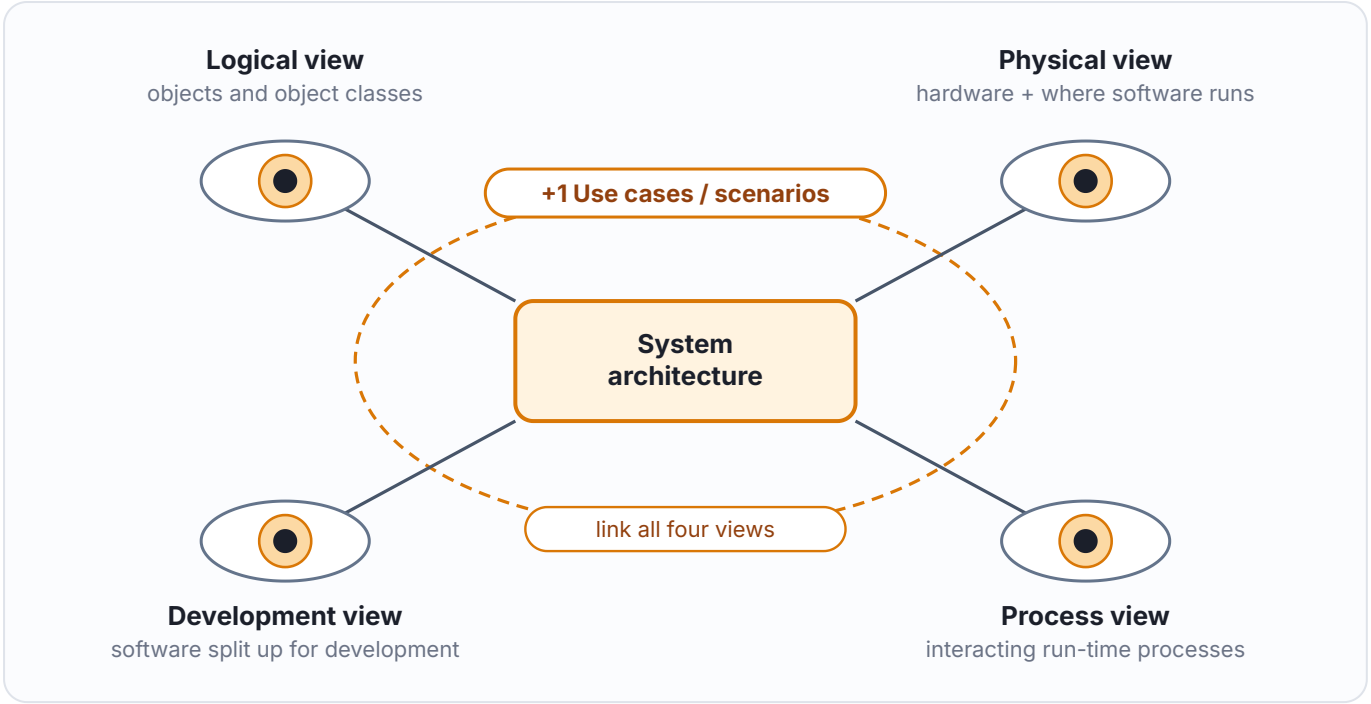


Fig Q2 Krutchen's 4+1 view model: four views of one system architecture, linked by use cases or scenarios (+1). Sommerville Fig 6.3 · Your slides, slides 18–19

View	What it shows	Useful for	Example: Mentcare (illustration)
1. Logical	The key abstractions in the system as **objects or object classes**. It should be possible to relate the system requirements to entities in this view.	Relating requirements to the design	Classes such as Patient, Consultation, Prescription, Clinic.
2. Process	How, **at run-time**, the system is composed of **interacting processes**.	Judging performance, availability and other non-functional characteristics	A client process on each clinic PC talks to a server process that handles patient-record requests.
3. Development	How the software is **decomposed for development**: its breakdown into components implemented by a single developer or development team.	Software managers and programmers	Modules for patient records, appointments and prescribing, each given to one team.
4. Physical	The **system hardware** and how software components are **distributed across the processors** in the system.	Systems engineers planning a system deployment	Clinic PCs connected over the network to a central database server.

+1 Use cases or scenarios. A scenario describes one typical interaction with the system, e.g. "a doctor records a consultation". Tracing the same scenario through all four views **links the views** and checks that they fit together.

In practice, conceptual views are almost always developed during design; other views are added as different aspects are discussed. It is rarely necessary to describe the system from all perspectives: except for critical systems, develop only the views that are useful for communication.

✎ EXAM TIP

Don't mix up the pairs: **process** view (run-time processes) ≠ **development** view (code split among teams); **physical** view (hardware) ≠ **logical** view (classes). The +1 links the four views; it is not a fifth view.

Q3 What are architectural design decisions? Explain the fundamental questions an architect must answer during architectural design.

5 MARKS | CO3 | L2 | VERY LIKELY

Marks split → Creative process 1 + Eight questions (diagram) 3 + Notes 1

MODEL ANSWER

Architectural design is a creative process in which you design a system organization that will satisfy the functional and non-functional requirements of a system.

1. **No formulaic process** — the process differs depending on the **type of system** being developed, the **background and experience** of the system architect, and the **specific requirements** for the system.
2. **A series of decisions** — so it is best to think of architectural design as a series of decisions to be made rather than a sequence of activities.
3. **Common decisions** — a number of common decisions span all design processes, and they affect the **non-functional characteristics** of the system. Based on their knowledge and experience, architects answer eight fundamental questions:

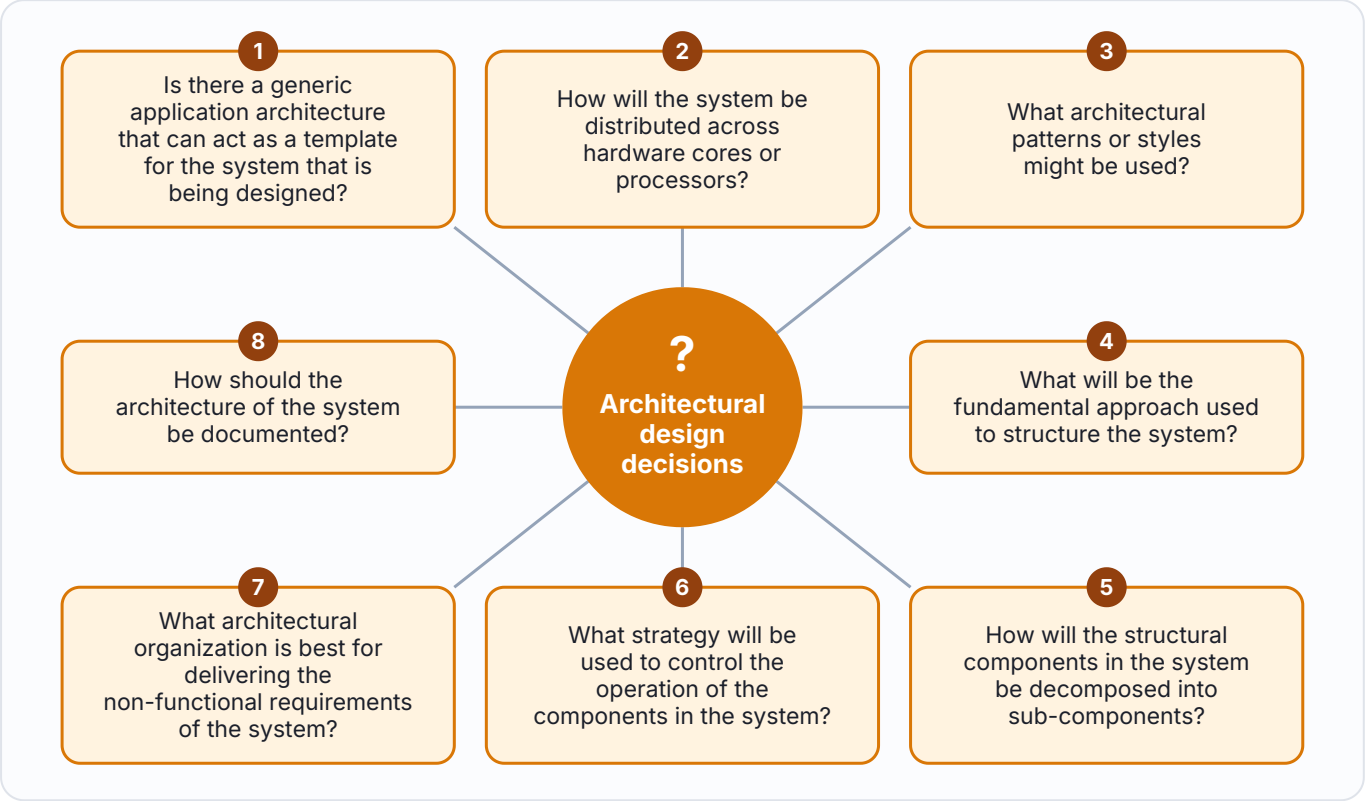


Fig Q3 Architectural design decisions: the eight fundamental questions (numbered clockwise). Sommerville Fig 6.2 · Your slides, slide 13

- **Distribution (2)** — embedded systems and apps for PCs and mobiles need not be distributed, but most large systems are; the choice of distribution architecture is a key decision that affects **performance and reliability**.

- **Structure, decomposition, control (4–6)** — together these form the **architectural style** (Garlan and Shaw): choose a structure such as client–server or layered, decide how to decompose components into sub-components, and in **control modelling** decide how the execution of components is controlled.
- **Evaluation** — the true test is how well the system meets its requirements when in use; before that, compare the design with reference architectures or generic patterns.

MEMORY TRICK

Generic · **D**istribution · **P**atterns · **S**tructure · **D**ecomposition · **C**ontrol · **N**on-functional · **D**ocumentation: "**G**ood **D**evelopers **P**refer **S**imple **D**esigns; **C**lear **N**on-functional **D**ocs".

Q4 Explain how architectural design depends on the non-functional requirements of a system (performance, security, safety, availability, maintainability).

5 MARKS · CO3 · L2 · VERY LIKELY

Marks split → Why NFRs drive the architecture 1 + Five strategies 3 + Conflict 1

MODEL ANSWER

Software architecture affects the **performance, robustness, distributability and maintainability** of a system (Bosch). Individual components implement the functional requirements, but **the dominant influence on the non-functional characteristics is the system's architecture**; non-functional requirements have the most significant effect on the architecture (Chen et al.). Therefore the **choice of architectural style and structure should depend on the non-functional requirements**:

If this is critical...	...use this architectural strategy	Why it works
1. Performance	**Localise critical operations** and minimise communications. Use **large** rather than fine-grain components.	Critical operations sit in a few components on the same computer, so most interactions happen inside a component. Replicating the system on different processors may also help.
2. Security	Use a **layered architecture** with the critical assets in the **inner layers**.	The most critical assets are protected by the innermost layers, with a high level of security validation applied to them.
3. Safety	**Localise safety-critical features** in a small number of sub-systems.	Reduces the cost of safety validation and may allow a protection system that safely shuts the system down on failure.
4. Availability	Include **redundant components** and mechanisms for **fault tolerance**.	Components can be replaced and updated without stopping the system; the system keeps working when a part fails.
5. Maintainability	Use **fine-grain, replaceable** components.	Small, self-contained components can readily be changed. Separate producers of data from consumers; avoid shared data structures.

Conflicts and trade-offs. There is potential conflict between some of these architectures. **Performance** wants a few **large** components, while **maintainability** wants many **small, fine-grain** components. If both are important, a compromise is found by using **different architectural patterns or styles for separate parts** of the system. Security is now almost always a critical requirement, so the architecture must maintain security while also satisfying the other non-functional requirements.

Example. In Mentcare, patient records are the critical asset, so they sit in the innermost layer; in the insulin pump, the safety-critical dose computation and checks are co-located in one small component that can be validated thoroughly.

> ✎ **EXAM TIP**
> Learn the five rows as keyword pairs: Performance → **large**, Security → **layers**, Safety → **localise**, Availability → **redundancy**, Maintainability → **fine-grain**. End with the performance vs maintainability conflict: it lifts a 4/5 answer to 5/5.

Q5 What are the advantages of designing and documenting the software architecture explicitly? Differentiate architecture in the small and architecture in the large.

5 MARKS | CO3 | L1 / L2 | LIKELY

Marks split → Three advantages 3 + Small vs large 2

MODEL ANSWER

(a) Advantages. "Explicit" means the architecture is deliberately designed and written down, not left to emerge by accident. Bass, Clements and Kazman suggest that explicitly designing and documenting a software architecture has three advantages:

1. **Stakeholder communication** — the architecture is a high-level presentation of the system that may be used as a **focus of discussion** by a range of different stakeholders (users, customers, managers, engineers).
2. **System analysis** — making the architecture explicit at an early stage requires some analysis, so we can check **whether the system can meet its non-functional requirements**. Architectural decisions have a profound effect on whether critical requirements such as performance, reliability and maintainability can be met.
3. **Large-scale reuse** — an architectural model is a compact, manageable description of how a system is organized and how its components interoperate. Systems with similar requirements often have the same architecture, so it **may be reused across a range of systems**; **product-line architectures** may be developed.

(b) Architecture in the small vs architecture in the large.

Point	Architecture in the small	Architecture in the large
Concerned with	The architecture of **individual programs**.	The architecture of **complex enterprise systems** that include other systems, programs and program components.
Main question	How an individual program is **decomposed into components**.	How many separate systems, programs and components are organized together.
Distribution and ownership	A single program.	May be **distributed over different computers**, which may be **owned and managed by different companies**.
Example (illustration)	One program split into user interface, calculations and data storage components.	Order and stock systems of one company linked to the payment and delivery systems of another.

MEMORY TRICK

Advantages = **CAR**: **C**ommunication (with stakeholders), **A**nalysis (of the system), **R**euse (large-scale). Small = one **program**; large = an **enterprise** of systems.

Q6 What are box-and-line diagrams? Explain the two ways in which architectural models are used.

5 MARKS | CO3 | L2 | LIKELY

Marks split → Definition + strengths 1 + Criticism 1 + Two uses 2 + When rigorous 1

MODEL ANSWER

Box-and-line (block) diagrams are simple, informal diagrams in which **boxes** represent components (entities) and **lines or arrows** show the relationships between them, e.g. the packing robot architecture. They are the **most frequently used** method for documenting software architectures.

1. **Simple and informal** — quick to draw, even on a whiteboard.
2. **Intuitive** — people from different disciplines (domain experts and engineers) can readily understand them and join the discussion.
3. **Not cluttered with detail** — they show the whole system, so they are useful for **communication with stakeholders** and for **project planning**.

Criticism. Bass et al. criticise them because they **lack semantics** (precise meaning) and are very abstract: they do not show the **types of relationships** between components, nor the **externally visible properties** of the entities or sub-systems.

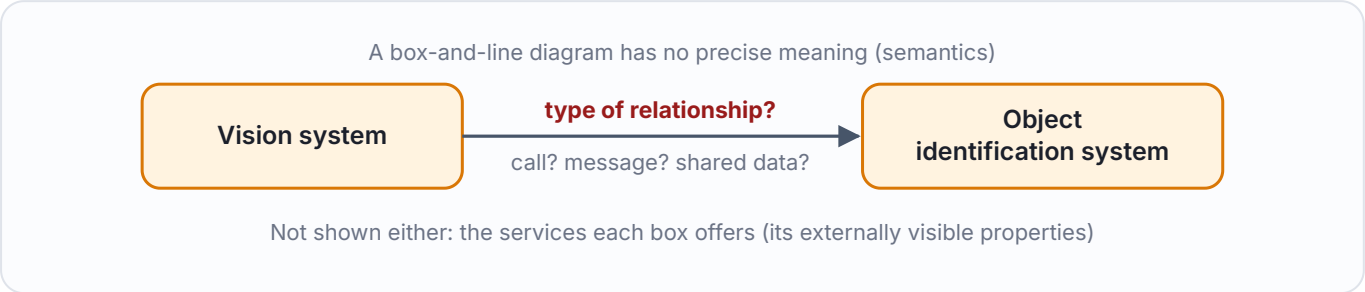


Fig Q6 Why box-and-line diagrams "lack semantics": the arrow and boxes leave the details unstated. Drawn for this guide

Two uses of architectural models. Whether the criticism matters depends on how the model is used; this explains why industry uses block diagrams although theory criticises them.

	1. To facilitate discussion about the system design	2. To document an architecture that has been designed
Aim	A high-level architectural view that is **not cluttered with detail**.	A **complete system model** showing the components, their **interfaces** and their **connections**.
Benefit	Stakeholders can relate to it and discuss the system as a whole; it identifies key components, so managers can assign people and plan development.	A detailed description makes the system easier to **understand and evolve**.
Notation	Informal block diagrams are enough.	Ideally a more rigorous notation, e.g. an ADL.

When is rigour worth it? A detailed, complete description leaves less scope for misunderstanding the relationships between components, but it is **expensive and time-consuming** to develop and it is hard to know whether it is cost-effective, so it is not widely used. It is mainly worthwhile for **critical systems**; otherwise, informal models useful for communication are enough.

> ✎ **EXAM TIP**
> Write the key sentence: "the requirements for model semantics depend on how the models are used". It resolves the contradiction and earns the last mark.

Q7 Explain architectural design with the example of the packing robot control system. Why must the architecture be designed early, even in agile development? Mention architecture reuse.

5 MARKS | CO3 | L2 | LIKELY

Marks split → Diagram 2 + Explanation 1 + Agility 1 + Reuse 1

MODEL ANSWER

Architectural design identifies the main structural components of a system and the relationships between them. Sommerville's example is a **packing robot** that can pack different kinds of objects. It uses a vision component to pick out objects on a conveyor, identify the type of object and select the right kind of packaging. It then moves objects from the delivery conveyor to be packaged and places packaged objects on another conveyor.

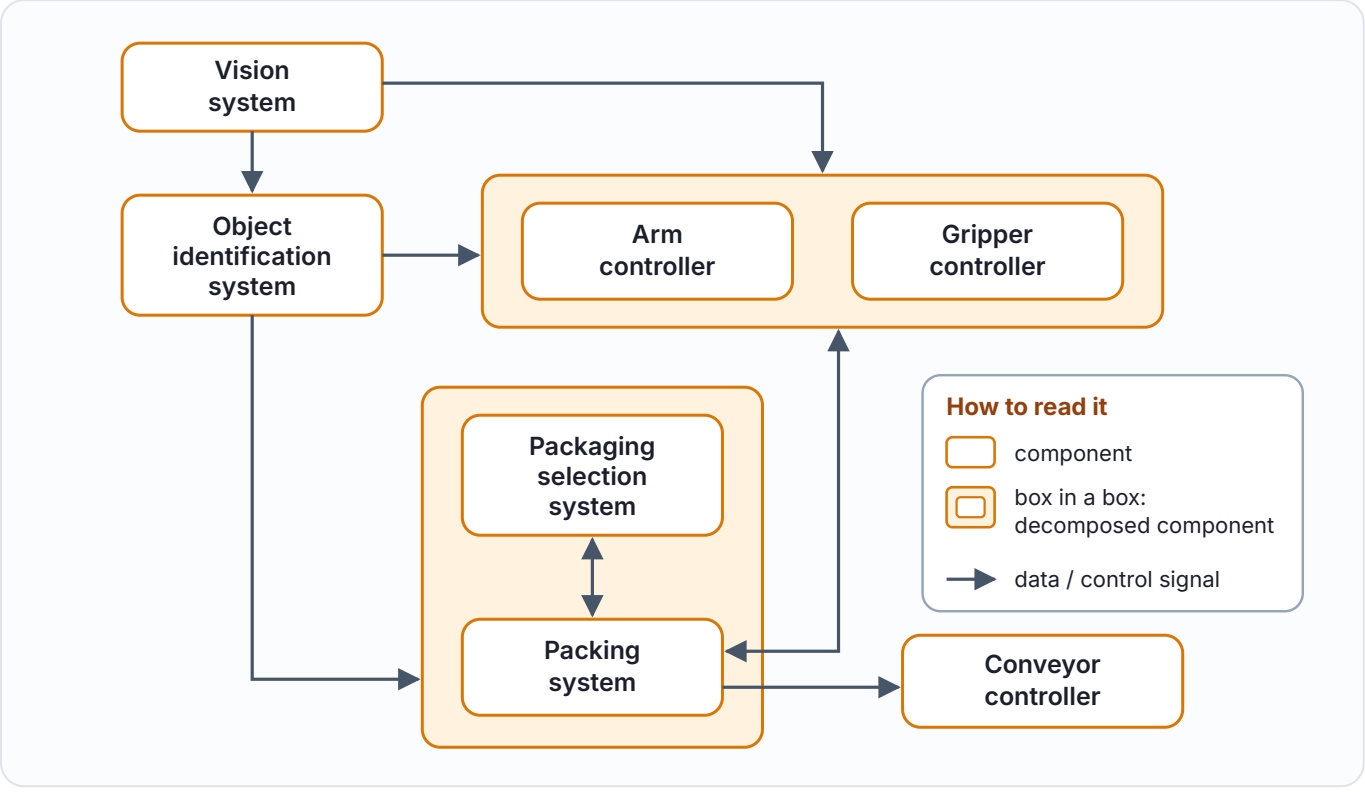


Fig Q7 The architecture of a packing robot control system (the legend box is not part of the answer). Sommerville Fig 6.1 · Your slides, slide 5

How to read it. This is an abstract **block (box-and-line) diagram**, a conceptual view of the system:

- **Boxes** represent components, e.g. the vision system and the conveyor controller.

- **Boxes within boxes** show a component decomposed into sub-components: arm + gripper controllers form one sub-system; packaging selection + packing systems form another.
- **Arrows** mean that data and/or control signals pass from component to component in the direction of the arrow.

Agility and architecture. It is generally accepted that an **early stage of agile processes** is to design an overall system architecture; incremental development of architectures is not usually successful. **Refactoring** individual components in response to change is relatively easy, but **refactoring the architecture is expensive**, because it affects so many components: most of them would have to be modified to adapt to the change.

Architecture reuse. Each system is unique, but architectures are often reused:

1. **Same domain** — systems in the same application domain often have similar architectures that reflect domain concepts.
2. **Product lines** — application product lines are built around a **core architecture** with **variants** that satisfy particular customer requirements.
3. **Patterns or styles** — the architecture may be designed around one or more architectural patterns or styles, which capture the essence of an architecture and can be instantiated in different ways.

> ✎ **EXAM TIP**
> Redraw the robot with its arrow directions and both shaded groups. For the agile part, the key line is: refactoring the architecture affects most components.

Backup questions (prepare if you have time)

5M **How should architectural views be represented? Compare the UML, ADLs and informal notations.**

Answer outline → **UML**: some argue it suits architecture; Sommerville disagrees (no abstractions for high-level description; object classes are too close to the implementation), so it is best for detailed documentation or model-driven development. **ADLs**: components and connectors with rules for well-formed architectures; precise but specialist, not widely used. **Informal notations**: quick, easy, the most common; except for critical systems, develop only the views useful for communication. Revise 6.10.

2M **What is a conceptual view? Give an example.**

Answer outline → Hofmeister et al.: an abstract view that can (1) be the basis for decomposing high-level requirements into detailed specifications, (2) help decide which components can be reused, (3) represent a product line rather than a single system. Example: the packing robot (Fig Q7). Almost always developed during design. Revise 6.9.

2M **Define an architectural pattern (style).**

Answer outline → A description of a system organization, such as a client–server organization or a layered architecture; it captures the essence of an architecture that has been used in different software systems. "Pattern" and "style" mean the same thing. (The patterns themselves are studied after CIE-1.) Revise 6.7.

5M **Why is architectural design called the critical link between requirements engineering and design?**

Answer outline → Definition → it is the first stage of design and identifies the main structural components and their relationships → output: an architectural model of communicating components → requirements and architecture overlap: an abstract architecture grouping functions into large-scale sub-systems is used to discuss requirements with stakeholders. Sketch study guide Fig 6.1 (Requirements engineering → Architectural design → Detailed design). Revise 6.1.

PRACTICE

Predicted CIE-1 Paper

Two practice papers in the same format as last year's: 30 marks, 1 hour, Q1 compulsory, then Q2 or Q3. Write them under exam conditions, then check against the model answers (each "Answer" link is clickable).

Practice paper A

Max marks: 30 · Time: 1 hour · Question 1 is mandatory · Answer any one of Question 2 or 3 · Each part carries 5 marks.

Q	Question	Marks	CO	Model answer
1a	Explain the waterfall model with a neat diagram. When is it appropriate, and what are its problems?	5	CO1	Unit 1 · Q3
1b	Differentiate between functional and non-functional requirements with examples.	5	CO2	Unit 2 · Q1
1c	Explain Krutchen's 4+1 view model of software architecture with a neat diagram.	5	CO3	Unit 3 · Q2
	2 OR 3			
2a	Explain the process improvement cycle and the SEI capability maturity levels with a neat diagram.	5	CO1	Unit 1 · Q6
2b	Explain the requirements elicitation and analysis process with a neat diagram. Briefly explain interviewing and ethnography.	5	CO2	Unit 2 · Q4
2c	Explain the design concepts: abstraction, modularity, information hiding and functional independence.	5	CO2	Unit 2 · Q7
3a	How do software processes cope with change? Explain prototyping and incremental delivery.	5	CO1	Unit 1 · Q5
3b	What is requirements validation? Explain the requirements checks and validation techniques.	5	CO2	Unit 2 · Q6
3c	Explain how architectural design depends on the non-functional requirements of a system.	5	CO3	Unit 3 · Q4

Practice paper B

Same rules. This one repeats last year's most-asked areas with fresh wording.

Q	Question	Marks	CO	Model answer
1a	Explain the ACM/IEEE Code of Ethics. What are the issues of professional responsibility?	5	CO1	Unit 1 · Q2

Q	Question	Marks	CO	Model answer
1b	Explain the structure of the IEEE software requirements specification document.	5	CO2	Unit 2 · Q5
1c	What are architectural design decisions? Explain the fundamental questions an architect must answer.	5	CO3	Unit 3 · Q3
	2 OR 3			
2a	With a neat diagram describe the incremental development model. What are its advantages and problems?	5	CO1	Unit 1 · Q4
2b	List the principles of agile methods and explain the Scrum sprint cycle with a neat labelled diagram.	5	CO1	Unit 1 · Q8
2c	Explain how the requirements model is translated into the design model, and describe the elements of the design model.	5	CO2	Unit 2 · Q8
3a	Describe the advantages of the XP practices: refactoring, pair programming and user stories.	5	CO1	Unit 1 · Q7
3b	Explain the requirements engineering process with a neat labelled spiral diagram.	5	CO2	Unit 2 · Q3
3c	What are non-functional requirements? List their types and the metrics for specifying them.	5	CO2	Unit 2 · Q2

✎ **HOW TO USE THE PRACTICE PAPERS**

Set a 60-minute timer. Do Q1 first (it's compulsory), then glance at Q2 and Q3 and pick the one where you can draw the most diagrams confidently. Leave 2 minutes at the end to re-read your answers.